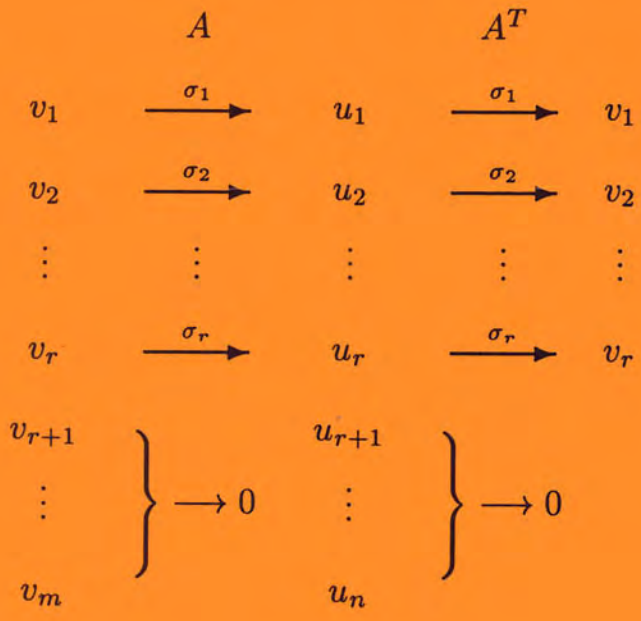
Pure and Applied Mathematics
A Wiley-Interscience Series of Texts, Monographs, and Tracts

# Fundamentals of Matrix Computations

*Second Edition*

*David S. Watkins*



# *Fundamentals of Matrix Computations*

*Second Edition*

**PURE AND APPLIED MATHEMATICS**

A Wiley-Interscience Series of Texts, Monographs, and Tracts

Founded by RICHARD COURANT
Editors: MYRON B. ALLEN III, DAVID A. COX, PETER LAX
Editors Emeriti: PETER HILTON, HARRY HOCHSTADT, JOHN TOLAND

A complete list of the titles in this series appears at the end of this volume.

# *Fundamentals of Matrix Computations*

*Second Edition*

**DAVID S. WATKINS**

WILEY-INTERSCIENCE

A JOHN WILEY & SONS, INC., PUBLICATION

This text is printed on acid-free paper. ♾



Published simultaneously in Canada.



For ordering and customer service, call 1-800-CALL WILEY.

***Library of Congress Cataloging-in-Publication Data is available.***

ISBN 0-471-21394-2

Printed in the United States of America

10 9 8 7 6 5 4 3 2

# Contents

*This page intentionally left blank*

# *Preface*

This book was written for advanced undergraduates, graduate students, and mature scientists in mathematics, computer science, engineering, and all disciplines in which numerical methods are used. At the heart of most scientific computer codes lie matrix computations, so it is important to understand how to perform such computations efficiently and accurately. This book meets that need by providing a detailed introduction to the fundamental ideas of numerical linear algebra.

The prerequisites are a first course in linear algebra and some experience with computer programming. For the understanding of some of the examples, especially in the second half of the book, the student will find it helpful to have had a first course in differential equations.

There are several other excellent books on this subject, including those by Demmel [15], Golub and Van Loan [33], and Trefethen and Bau [71]. Students who are new to this material often find those books quite difficult to read. The purpose of this book is to provide a gentler, more gradual introduction to the subject that is nevertheless mathematically solid. The strong positive student response to the first edition has assured me that my first attempt was successful and encouraged me to produce this updated and extended edition.

The first edition was aimed mainly at the undergraduate level. As it turned out, the book also found a great deal of use as a graduate text. I have therefore added new material to make the book more attractive at the graduate level. These additions are detailed below. However, the text remains suitable for undergraduate use, as the elementary material has been kept largely intact, and more elementary exercises have been added. The instructor can control the level of difficulty by deciding which

sections to cover and how far to push into each section. Numerous advanced topics are developed in exercises at the ends of the sections.

The book contains many exercises, ranging from easy to moderately difficult. Some are interspersed with the textual material and others are collected at the end of each section. Those that are interspersed with the text are meant to be worked immediately by the reader. This is my way of getting students actively involved in the learning process. In order to get something out, you have to put something in. Many of the exercises at the ends of sections are lengthy and may appear intimidating at first. However, the persistent student will find that s/he can make it through them with the help of the ample hints and advice that are given. I encourage every student to work as many of the exercises as possible.

### Numbering Scheme

Nearly all numbered items in this book, including theorems, lemmas, numbered equations, examples, and exercises, share a single numbering scheme. For example, the first numbered item in Section 1.3 is Theorem 1.3.1. The next two numbered items are displayed equations, which are numbered (1.3.2) and (1.3.3), respectively. These are followed by the first exercise of the section, which bears the number 1.3.4. Thus each item has a unique number: the only item in the book that has the number 1.3.4 is Exercise 1.3.4. Although this scheme is unusual, I believe that most readers will find it perfectly natural, once they have gotten used to it. Its big advantage is that it makes things easy to find: The reader who has located Exercises 1.4.15 and 1.4.25 but is looking for Example 1.4.20, knows for sure that this example lies somewhere between the two exercises.

There are a couple of exceptions to the scheme. For technical reasons related to the type setting, tables and figures (the so-called *floating bodies*) are numbered separately by chapter. For example, the third figure of Chapter 1 is Figure 1.3.

### New Features of the Second Edition

### Use of MATLAB

By now MATLAB[1] is firmly established as the most widely used vehicle for teaching matrix computations. MATLAB is an easy to use, very high-level language that allows the student to perform much more elaborate computational experiments than before. MATLAB is also widely used in industry. I have therefore added many examples and exercises that make use of MATLAB. This book is not, however, an introduction to MATLAB, nor is it a MATLAB manual. For those purposes there are other books available, for example, the *MATLAB Guide* by Higham and Higham [40].

[1]MATLAB is a registered trademark of the MathWorks, Inc. (`http://www.mathworks.com`)

However, MATLAB's extensive help facilities are good enough that the reader may feel no need for a supplementary text. In an effort to make it easier for the student to use MATLAB with this book, I have included an index of MATLAB terms, separate from the ordinary index.

I used to make my students write and debug their own Fortran programs. I have left the Fortran exercises from the first edition largely intact. I hope a few students will choose to work through some of these worthwhile projects.

### More Applications

In order to help the student better understand the importance of the subject matter of this book, I have included more examples and exercises on applications (solved using MATLAB), mostly at the beginnings of chapters. I have chosen very simple applications: electrical circuits, mass-spring systems, simple partial differential equations. In my opinion the simplest examples are the ones from which we can learn the most.

### Earlier Introduction of the Singular Value Decomposition (SVD)

The SVD is one of the most important tools in numerical linear algebra. In the first edition it was placed in the final chapter of the book, because it is impossible to discuss methods for computing the SVD until after eigenvalue problems have been discussed. I have since decided that the SVD needs to be introduced sooner, so that the student can find out earlier about its properties and uses. With the help of MATLAB, the student can experiment with the SVD without knowing anything about how it is computed. Therefore I have added a brief chapter on the SVD in the middle of the book.

### New Material on Iterative Methods

The biggest addition to the book is a chapter on iterative methods for solving large, sparse systems of linear equations. The main focus of the chapter is the powerful conjugate-gradient method for solving symmetric, positive definite systems. However, the classical iterations are also discussed, and so are preconditioners. Krylov subspace methods for solving indefinite and nonsymmetric problems are surveyed briefly.

There are also two new sections on methods for solving large, sparse eigenvalue problems. The discussion includes the popular implicitly-restarted Arnoldi and Jacobi-Davidson methods.

I hope that these additions in particular will make the book more attractive as a graduate text.

### Other New Features

To make the book more versatile, a number of other topics have been added, including:

- a backward error analysis of Gaussian elimination, including a discussion of the modern componentwise error analysis.
- a discussion of reorthogonalization, a practical means of obtaining numerically orthonormal vectors.
- a discussion of how to update the $QR$ decomposition when a row or column is added to or deleted from the data matrix, as happens in signal processing and data analysis applications.
- a section introducing new methods for the symmetric eigenvalue problem that have been developed since the first edition was published.

A few topics have been deleted on the grounds that they are either obsolete or too specialized. I have also taken the opportunity to correct several vexing errors from the first edition. I can only hope that I have not introduced too many new ones.

DAVID S. WATKINS

*Pullman, Washington*
*January, 2002*

# *Acknowledgments*

I am greatly indebted to the authors of some of the early works in this field. These include A. S. Householder [43], J. H. Wilkinson [81], G. E. Forsythe and C. B. Moler [24], G. W. Stewart [67], C. L. Lawson and R. J. Hanson [48], B. N. Parlett [54], A. George and J. W. Liu [30], as well as the authors of the Handbook [83], the EISPACK Guide [64], and the LINPACK Users' Guide [18]. All of them influenced me profoundly. By the way, every one of these books is still worth reading today. Special thanks go to Cleve Moler for inventing MATLAB, which has changed everything.

Most of the first edition was written while I was on leave, at the University of Bielefeld, Germany. I am pleased to thank once again my host and longtime friend, Ludwig Elsner. During that stay I received financial support from the Fulbright commission. A big chunk of the second edition was also written in Germany, at the Technical University of Chemnitz. I thank my host (and another longtime friend), Volker Mehrmann. On that visit I received financial support from Sonderforschungsbereich 393, TU Chemnitz. I am also indebted to my home institution, Washington State University, for its support of my work on both editions.

I thank once again professors Dale Olesky, Kemble Yates, and Tjalling Ypma, who class-tested a preliminary version of the first edition. Since publication of the first edition, numerous people have sent me corrections, feedback, and comments. These include A. Cline, L. Dieci, E. Jessup, D. Koya, D. D. Olesky, B. N. Parlett, A. C. Raines, A. Witt, and K. Wright. Finally, I thank the many students who have helped me learn how to teach this material over the years.

D. S. W.

*This page intentionally left blank*

# 1

# *Gaussian Elimination and Its Variants*

One of the most frequently occurring problems in all areas of scientific endeavor is that of solving a system of $n$ linear equations in $n$ unknowns. For example, in Section 1.2 we will see how to compute the voltages and currents in electrical circuits, analyze simple elastic systems, and solve differential equations numerically, all by solving systems of linear equations. The main business of this chapter is to study the use of Gaussian elimination to solve such systems. We will see that there are many ways to organize this fundamental algorithm.

## 1.1 MATRIX MULTIPLICATION

Before we begin to study the solution of linear systems, let us take a look at some simpler matrix computations. In the process we will introduce some of the basic themes that will appear throughout the book. These include the use of operation counts (flop counts) to measure the complexity of an algorithm, the use of partitioned matrices and block matrix operations, and an illustration of the wide variety of ways in which a simple matrix computation can be organized.

### Multiplying a Matrix by a Vector

Of the various matrix operations, the most fundamental one that cannot be termed entirely trivial is that of multiplying a matrix by a vector. Consider an $n \times m$ matrix,

that is, a matrix with $n$ rows and $m$ columns:

$$A = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1m} \\ a_{21} & a_{22} & \cdots & a_{2m} \\ \vdots & \vdots & & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nm} \end{bmatrix}.$$

The entries of $A$ might be real or complex numbers. Let us assume that they are real for now. Given an $m$-tuple $x$ of real numbers:

$$x = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_m \end{bmatrix},$$

we can multiply $A$ by $x$ to get a product $b = Ax$, where $b$ is an $n$-tuple. Its $i$th component is given by

$$b_i = a_{i1}x_1 + a_{i2}x_2 + \cdots + a_{im}x_m = \sum_{j=1}^{m} a_{ij}x_j. \tag{1.1.1}$$

In words, the $i$th component of $b$ is obtained by taking the inner product (dot product) of $i$th row of $A$ with $x$.

**Example 1.1.2** An example of matrix-vector multiplication with $n = 2$ and $m = 3$ is

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \begin{bmatrix} 7 \\ 8 \\ 9 \end{bmatrix} = \begin{bmatrix} 50 \\ 122 \end{bmatrix},$$

since $1 \times 7 + 2 \times 8 + 3 \times 9 = 50$ and $4 \times 7 + 5 \times 8 + 6 \times 9 = 122$. □

A computer code to perform matrix-vector multiplication might look something like this:

$$\begin{array}{l} b \leftarrow 0 \\ \text{for } i = 1, \ldots, n \\ \quad \left[ \begin{array}{l} \text{for } j = 1, \ldots, m \\ \quad \left[ \; b_i \leftarrow b_i + a_{ij}x_j \right. \end{array} \right. \end{array} \tag{1.1.3}$$

The $j$-loop accumulates the inner product $b_i$.

There is another way to view matrix-vector multiplication that turns out to be quite useful. Take another look at (1.1.1), but now view it as a formula for the entire vector $b$ rather than just a single component. In other words, take the equation (1.1.1), which is really $n$ equations for $b_1, b_2, \ldots b_n$, and stack the $n$ equations into a single vector

equation.

$$\begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_n \end{bmatrix} = \begin{bmatrix} a_{11} \\ a_{21} \\ \vdots \\ a_{n1} \end{bmatrix} x_1 + \begin{bmatrix} a_{12} \\ a_{22} \\ \vdots \\ a_{n2} \end{bmatrix} x_2 + \cdots + \begin{bmatrix} a_{1m} \\ a_{2m} \\ \vdots \\ a_{nm} \end{bmatrix} x_m. \tag{1.1.4}$$

This shows that $b$ is a linear combination of the columns of $A$.

**Example 1.1.5** Referring to Example 1.1.2, we have

$$\begin{bmatrix} 50 \\ 122 \end{bmatrix} = \begin{bmatrix} 1 \\ 4 \end{bmatrix} 7 + \begin{bmatrix} 2 \\ 5 \end{bmatrix} 8 + \begin{bmatrix} 3 \\ 6 \end{bmatrix} 9.$$

□

**Proposition 1.1.6** *If $b = Ax$, then $b$ is a linear combination of the columns of $A$.*

If we let $A_j$ denote the $j$th column of $A$, we have

$$b = \sum_{j=1}^{m} A_j x_j.$$

Expressing these operations as computer pseudocode, we have

$$\begin{array}{l} b \leftarrow 0 \\ \text{for } j = 1, \ldots, m \\ \quad \left[ \; b \leftarrow b + A_j x_j \right. \end{array}$$

If we use a loop to perform each vector operation, the code becomes

$$\begin{array}{l} b \leftarrow 0 \\ \text{for } j = 1, \ldots, m \\ \quad \left[ \begin{array}{l} \text{for } i = 1, \ldots, n \\ \quad \left[ \; b_i \leftarrow b_i + a_{ij} x_j \right. \end{array} \right. \end{array} \tag{1.1.7}$$

Notice that (1.1.7) is identical to (1.1.3), except that the loops are interchanged. The two algorithms perform exactly the same operations but not in the same order. We call (1.1.3) a *row-oriented* matrix-vector multiply, because it accesses $A$ by rows. In contrast, (1.1.7) is a *column-oriented* matrix-vector multiply.

## Flop Counts

Real numbers are normally stored in computers in a floating-point format. The arithmetic operations that a computer performs on these numbers are called floating-point operations or *flops*, for short. The update $b_i \leftarrow b_i + a_{ij} x_j$ involves two flops, one floating-point multiply and one floating-point add.[1]

[1] We discuss floating-point arithmetic in Section 2.5.

Any time we run a computer program, we want to know how long it will take to complete its task. If the job involves large matrices, say $1000 \times 1000$, it may take a while. The traditional way to estimate running time is to count how many flops the computer must perform. Let us therefore count the flops in a matrix-vector multiply. Looking at (1.1.7), we see that if $A$ is $n \times m$, the outer loop will be executed $m$ times. On each of these passes, the inner loop is executed $n$ times. Each execution of the inner loop involves two flops. Therefore the total flop count for the algorithm is $2nm$. This is a particularly easy flop count. On more complex algorithms it will prove useful to use the following device. Replace each loop by a summation sign $\Sigma$. Since the inner loop is executed for $i = 1, \ldots, n$, and there are two flops per pass, the total number of flops performed on each execution of the inner loop is $\sum_{i=1}^{n} 2$. Since the outer loop runs from $j = 1 \ldots m$, the total number of flops is

$$\sum_{j=1}^{m} \sum_{i=1}^{n} 2 = \sum_{j=1}^{m} 2n = 2nm.$$

The flop count gives a rough idea of how long it will take the algorithm to run. Suppose, for example, we have run the algorithm on a $300 \times 400$ matrix and observed how long it takes. If we now want to run the same algorithm on a matrix of size $600 \times 400$, we expect it to take about twice as long, since we are doubling $n$ while holding $m$ fixed and thereby doubling the flop count $2nm$.

Square matrices arise frequently in applications. If $A$ is $n \times n$, the flop count for a matrix-vector multiply is $2n^2$. If we have performed a matrix-vector multiply on, say, a $500 \times 500$ matrix, we can expect the same operation on a $1000 \times 1000$ matrix to take about four times as long, since doubling $n$ quadruples the flop count in this case.

An $n \times n$ matrix-vector multiply is an example of what we call an $O(n^2)$ process, or a process of *order* $n^2$. This just means that the amount of work is proportional to $n^2$. The notation is used as a way of emphasizing the dependence on $n$ and de-emphasizing the proportionality constant, which is 2 in this case. Any $O(n^2)$ process has the property that doubling the problem size quadruples the amount of work.

It is important to realize that the flop count is only a crude indicator of the amount of work that an algorithm entails. It ignores many other tasks that the computer must perform in the course of executing the algorithm. The most important of these are fetching data from memory to be operated on and returning data to memory once the operations are done. On many computers the memory access operations are slower than the floating point operations, so it makes more sense to count memory accesses than flops. The flop count is useful, nevertheless, because it also gives us a rough count of the memory traffic. For each operation we must fetch operands from memory, and after each operation we must return the result to memory. This is a gross oversimplification of what actually goes on in modern computers. The execution speed of the algorithm can be affected drastically by how the memory traffic is organized. We will have more to say about this at the end

of the section. Nevertheless, the flop count gives us a useful first indication of an algorithm's operation time, and we shall count flops as a matter of course.

**Exercise 1.1.8** Begin to familiarize yourself with MATLAB. Log on to a machine that has MATLAB installed, and start MATLAB. From MATLAB's command line type `A = randn(3,4)` to generate a $3 \times 4$ matrix with random entries. To learn more about the `randn` command, type `help randn`. Now type `x = randn(4,1)` to get a vector (a $4 \times 1$ matrix) of random numbers. To multiply $A$ by $x$ and store the result in a new vector $b$, type `b = A*x`.

To get MATLAB to save a transcript of your session, type `diary on`. This will cause a file named *diary*, containing a record of your MATLAB session, to be saved. Later on you can edit this file, print it out, turn it in to your instructor, or whatever. To learn more about the `diary` command, type `help diary`.

Other useful commands are `help` and `help help`. To see a demonstration of MATLAB's capabilities, type `demo`. □

**Exercise 1.1.9** Consider the following simple MATLAB program.

```
n = 200;
for jay = 1:4
  if jay > 1
    oldtime = time;
  end
  A = randn(n);
  x = randn(n,1);
  t = cputime;
  b = A*x;
  matrixsize = n
  time = cputime - t
  if jay > 1
    ratio = time/oldtime
  end
  n = 2*n;
end
```

The syntax is simple enough that you can readily figure out what the program does. The commands `randn` and `b = A*x` are familiar from the previous exercise. The function `cputime` tells how much computer (central processing unit) time the current MATLAB session has used. This program times the execution of a matrix-vector multiply for square matrices $A$ of dimension 200, 400, 800, and 1600.

Enter this program into a file called matvectime.m. Actually, you can call it whatever you please, but you must use the .m extension. (MATLAB programs are called m-files.) Now start MATLAB and type `matvectime` (without the .m) to execute the program. Depending on how fast your computer is, you may like to change the size of the matrix or the number of times the `jay` loop is executed. If the computer is fast and has a relatively crude clock, it might say that the execution time is zero, depending on how big a matrix you start with.

MATLAB normally prints the output of all of its operations to the screen. You can suppress printing by terminating the operation with a semicolon. Looking at the program, we see that `A`, `x`, `t`, and `b` will not be printed, but `matrixsize`, `time`, and `ratio` will.

Look at the values of `ratio`. Are they close to what you would expect based on the flop count? □

**Exercise 1.1.10** Write a MATLAB program that performs matrix-vector multiplication two different ways: (a) using the built-in MATLAB command `b = A*x`, and (b) using loops, as follows.

```
for j = 1:n
  for i = 1:n
    b(i) = A(i,j)*x(j);
  end
end
```

Time the two different methods on matrices of various sizes. Which method is faster? (You may want to use some of the code from Exercise 1.1.9.) □

**Exercise 1.1.11** Write a Fortran or C program that performs matrix-vector multiplication using loops. How does its speed compare with that of MATLAB? □

### Multiplying a Matrix by a Matrix

If $A$ is an $n \times m$ matrix, and $X$ is $m \times p$, we can form the product $B = AX$, which is $n \times p$. The $(i, j)$ entry of $B$ is

$$b_{ij} = \sum_{k=1}^{m} a_{ik} x_{kj}. \tag{1.1.12}$$

In words, the $(i, j)$ entry of $B$ is the dot product of the $i$th row of $A$ with the $j$th column of $X$. If $p = 1$, this operation reduces to matrix-vector multiplication. If $p > 1$, the matrix-matrix multiply amounts to $p$ matrix-vector multiplies: the $j$th column of $B$ is just $A$ times the $j$th column of $X$.

A computer program to multiply $A$ by $X$ might look something like this:

$$\begin{array}{l} B \leftarrow 0 \\ \text{for } i = 1, \ldots, n \\ \quad \left[ \begin{array}{l} \text{for } j = 1, \ldots, p \\ \quad \left[ \begin{array}{l} \text{for } k = 1, \ldots, m \\ \quad \left[ \; b_{ij} \leftarrow b_{ij} + a_{ik} x_{kj} \right. \end{array} \right. \end{array} \right. \end{array} \tag{1.1.13}$$

The decision to put the $i$-loop outside the $j$-loop was perfectly arbitrary. In fact, the order in which the updates $b_{ij} \leftarrow b_{ij} + a_{ik} x_{kj}$ are made is irrelevant, so the three loops can be nested in any way. Thus there are six basic variants of the matrix-matrix multiplication algorithm. These are all equivalent in principle. In practice, some

versions may run faster than others on a given computer, because of the order in which the data are accessed.

It is a simple matter to count the flops in matrix-matrix multiplication. Since there are two flops in the innermost loop of (1.1.13), the total flop count is

$$\sum_{i=1}^{n}\sum_{j=1}^{p}\sum_{k=1}^{m} 2 = 2nmp.$$

In the important case when all of the matrices are square of dimension $n \times n$, the flop count is $2n^3$. Thus square matrix multiplication is an $O(n^3)$ operation. This function grows rather quickly with $n$: each time $n$ is doubled, the flop count is multiplied by eight. (However, this is not the whole story. See the remarks on fast matrix multiplication at the end of this section.)

**Exercise 1.1.14** Modify the MATLAB code from Exercise 1.1.9 by changing the matrix-vector multiply `b = A*x` to a matrix-matrix multiply `B = A*X`, where $X$ is $n \times n$. You may also want to decrease the initial matrix dimension $n$. Run the code and check the ratios. Are they close to what you would expect them to be, based on the flop count? □

### Block Matrices and Block Matrix Operations

The idea of partitioning matrices into blocks is simple but powerful. It is a useful tool for proving theorems, constructing algorithms, and developing faster variants of algorithms. We will use block matrices again and again throughout the book.

Consider the matrix product $AX = B$, where the matrices have dimensions $n \times m$, $m \times p$, and $n \times p$, respectively. Suppose we partition $A$ into blocks:

$$A = \begin{array}{c} \\ n_1 \\ n_2 \end{array} \begin{array}{c} \begin{array}{cc} m_1 & m_2 \end{array} \\ \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \end{array} \qquad \begin{cases} n_1 + n_2 = n \\ m_1 + m_2 = m. \end{cases} \tag{1.1.15}$$

The labels $n_1$, $n_2$, $m_1$, and $m_2$ indicate that the block $A_{ij}$ has dimensions $n_i \times m_j$. We can partition $X$ similarly.

$$X = \begin{array}{c} \\ m_1 \\ m_2 \end{array} \begin{array}{c} \begin{array}{cc} p_1 & p_2 \end{array} \\ \begin{bmatrix} X_{11} & X_{12} \\ X_{21} & X_{22} \end{bmatrix} \end{array} \qquad \begin{cases} m_1 + m_2 = m \\ p_1 + p_2 = p. \end{cases} \tag{1.1.16}$$

The numbers $m_1$ and $m_2$ are the same as in (1.1.15). Thus, for example, the number of rows in $X_{12}$ is the same as the number of columns in $A_{11}$ and $A_{21}$. Continuing in the same spirit, we partition $B$ as follows:

$$B = \begin{array}{c} \\ n_1 \\ n_2 \end{array} \begin{array}{c} \begin{array}{cc} p_1 & p_2 \end{array} \\ \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix} \end{array} \qquad \begin{cases} n_1 + n_2 = n \\ p_1 + p_2 = p. \end{cases} \tag{1.1.17}$$

The row partition of $B$ is the same as that of $A$, and the column partition is the same as that of $X$. The product $AX = B$ can now be written as

$$\begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \begin{bmatrix} X_{11} & X_{12} \\ X_{21} & X_{22} \end{bmatrix} = \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}. \tag{1.1.18}$$

We know that $B$ is related to $A$ and $X$ by the equations (1.1.12), but how are the blocks of $B$ related to the blocks of $A$ and $X$? We would hope to be able to multiply the blocks as if they were numbers. For example, we hope that $A_{11}X_{11} + A_{12}X_{21} = B_{11}$. Theorem 1.1.19 states that this is indeed the case.

**Theorem 1.1.19** *Let $A$, $X$, and $B$ be partitioned as in (1.1.15), (1.1.16), and (1.1.17), respectively. Then $AX = B$ if and only if*

$$A_{i1}X_{1j} + A_{i2}X_{2j} = B_{ij}, \qquad i, j = 1, 2.$$

You can easily convince yourself that Theorem 1.1.19 is true. It follows more or less immediately from the definition of matrix multiplication. We will skip the tedious but routine exercise of writing out a detailed proof. You might find the following exercise useful.

**Exercise 1.1.20** Consider matrices $A$, $X$, and $B$, partitioned as indicated.

$$A = \left[\begin{array}{c|cc} 1 & 3 & 2 \\ \hline 2 & 1 & 1 \\ -1 & 0 & 1 \end{array}\right] \qquad X = \left[\begin{array}{c|cc} 1 & 0 & 1 \\ \hline 2 & 1 & 1 \\ -1 & 2 & 0 \end{array}\right] \qquad B = \left[\begin{array}{c|cc} 5 & 7 & 4 \\ \hline 3 & 3 & 3 \\ -2 & 2 & -1 \end{array}\right]$$

Thus, for example, $A_{12} = \begin{bmatrix} 3 & 2 \\ 1 & 1 \end{bmatrix}$ and $A_{21} = \begin{bmatrix} -1 \end{bmatrix}$. Show that $AX = B$ and $A_{i1}X_{1j} + A_{i2}X_{2j} = B_{ij}$ for $i, j = 1, 2$. □

Once you believe Theorem 1.1.19, you should have no difficulty with the following generalization. Make a finer partition of $A$ into $r$ block rows and $s$ block columns.

$$A = \begin{array}{c} \\ n_1 \\ \vdots \\ n_r \end{array} \begin{array}{c} \begin{array}{ccc} m_1 & \cdots & m_s \end{array} \\ \begin{bmatrix} A_{11} & \cdots & A_{1s} \\ \vdots & & \vdots \\ A_{r1} & \cdots & A_{rs} \end{bmatrix} \end{array} \qquad \begin{cases} n_1 + \cdots + n_r = n \\ m_1 + \cdots + m_s = m. \end{cases} \tag{1.1.21}$$

Then partition $X$ *conformably* with $A$; that is, make the block row structure of $X$ identical to the block column structure of $A$.

$$X = \begin{array}{c} \\ m_1 \\ \vdots \\ m_s \end{array} \begin{array}{c} \begin{array}{ccc} p_1 & \cdots & p_t \end{array} \\ \begin{bmatrix} X_{11} & \cdots & X_{1t} \\ \vdots & & \vdots \\ X_{s1} & \cdots & X_{st} \end{bmatrix} \end{array} \qquad \begin{cases} m_1 + \cdots + m_s = m \\ p_1 + \cdots + p_t = p. \end{cases} \tag{1.1.22}$$

Finally, partition the product $B$ conformably with both $A$ and $X$.

$$B = \begin{array}{c} \\ n_1 \\ \vdots \\ n_r \end{array} \begin{array}{c} \begin{array}{ccc} p_1 & \cdots & p_t \end{array} \\ \begin{bmatrix} B_{11} & \cdots & B_{1t} \\ \vdots & & \vdots \\ B_{r1} & \cdots & B_{rt} \end{bmatrix} \end{array} \qquad \begin{cases} n_1 + \cdots + n_r = n \\ p_1 + \cdots + p_t = p. \end{cases} \tag{1.1.23}$$

**Theorem 1.1.24** *Let $A$, $X$, and $B$ be partitioned as in (1.1.21), (1.1.22), and (1.1.23), respectively. Then $B = AX$ if and only if*

$$B_{ij} = \sum_{k=1}^{s} A_{ik} X_{kj} \qquad i = 1, \ldots, r, \ j = 1, \ldots, t.$$

**Exercise 1.1.25** Make a partition of the matrix-vector product $Ax = b$ that demonstrates that $b$ is a linear combination of the columns of $A$. □

### Use of Block Matrix Operations to Decrease Data Movement

Suppose we wish to multiply $A$ by $X$ to obtain $B$. For simplicity, let us assume that the matrices are square, $n \times n$, although the idea to be discussed in this section can be applied to non-square matrices as well. Assume further that $A$ can be partitioned into $s$ block rows and $s$ block columns, where each block is $r \times r$.

$$A = \begin{bmatrix} A_{11} & \cdots & A_{1s} \\ \vdots & & \vdots \\ A_{s1} & \cdots & A_{ss} \end{bmatrix}.$$

Thus $n = rs$. We partition $X$ and $B$ in the same way. The assumption that the blocks are all the same size is again just for simplicity. (In practice we want nearly all of the blocks to be approximately square and nearly all of approximately the same size.) Writing a block version of (1.1.13), we have

$$\begin{array}{l} B \leftarrow 0 \\ \text{for } i = 1, \ldots, s \\ \quad \left[ \begin{array}{l} \text{for } j = 1, \ldots, s \\ \quad \left[ \begin{array}{l} \text{for } k = 1, \ldots, s \\ \quad \left[ \ B_{ij} \leftarrow B_{ij} + A_{ik} X_{kj} \right. \end{array} \right. \end{array} \right. \end{array} \tag{1.1.26}$$

A computer program based on this layout would perform the following operations repeatedly: grab the blocks $A_{ik}$, $X_{kj}$, and $B_{ij}$, multiply $A_{ik}$ by $X_{kj}$ and add the result to $B_{ij}$, using something like (1.1.13), then set $B_{ij}$ aside.

Varying the block size will not affect the total flop count, which will always be $2n^3$, but it can affect the performance of the algorithm dramatically nevertheless, because of the way the data are handled. Every computer has a memory hierarchy. This has always been the case, although the details have changed with time. Nowadays a typical computer has a small number of registers, a smallish, fast cache, a much larger, slower, main memory, and even larger, slower, bulk storage areas (e.g. disks and tapes). Data stored in the main memory must first be moved to cache and then to registers before it can be operated on. The transfer from main memory to cache is much slower than that from cache to registers, and it is also slower than the rate at which the computer can perform arithmetic. If we can move the entire arrays $A$, $X$, and $B$ into cache, then perform all of the operations, then move the result, $B$, back to main memory, we expect the job to be done much more quickly than if we must repeatedly move data back and forth. Indeed, if we can fit all of the arrays into cache, the total number of data transfers between slow and fast memory will be about $4n^2$, whereas the total number of flops is $2n^3$. Thus the ratio of arithmetic to memory transfers is $\frac{1}{2}n$ flops per data item, which implies that the relative importance of the data transfers decreases as $n$ increases. Unfortunately, unless $n$ is quite small, the cache will not be large enough to hold the entire matrices. Then it becomes beneficial to perform the matrix-matrix multiply by blocks.

Before we consider blocking, let us see what happens if we do not use blocks. Let us suppose the cache is big enough to hold two matrix columns or rows. Computation of entry $b_{ij}$ requires the $i$th row of $A$ and the $j$th column of $X$. The time required to move these into cache is proportional to $2n$, the number of data items. Once these are in fast memory, we can quickly perform the $2n$ flops. If we now want to calculate $b_{i,j+1}$, we can keep the $i$th row of $A$ in cache, but we need to bring in column $j+1$ of $X$, which means we have a time delay proportional to $n$. We can then perform the $2n$ flops to get $b_{i,j+1}$. The ratio of arithmetic to data transfers is about 2 flops per data item. In other words, the number of transfers between main memory and cache is proportional to the amount of arithmetic done. This severely limits performance. There are many ways to reorganize the matrix-matrix multiplication algorithm (1.1.13) without blocking, but they all suffer from this same limitation.

Now let us see how we can improve the situation by using blocks. Suppose we perform algorithm (1.1.26) using a block size $r$ that is small enough that the three blocks $A_{ik}$, $X_{kj}$, and $B_{ij}$ can all fit into cache at once. The time to move these blocks from main memory to cache is proportional to $3r^2$, the number of data items. Once they are in the fast memory, the $2r^3$ flops that the matrix-matrix multiply requires can be performed relatively quickly. The number of flops per data item is $\frac{2}{3}r$, which tells us that we can maximize the ratio of arithmetic to data transfers by making $r$ as large as possible, subject to the constraint that three blocks must fit into cache at once. The ratio $\frac{2}{3}r$ can be improved upon by intelligent handling of the block $B_{ij}$, but the main point is that the ratio of arithmetic to data transfers is $O(r)$. The larger the blocks are, the less significant the data transfers become. The reason for this is simply that $O(r^3)$ flops are performed on $O(r^2)$ data items.

We also remark that if the computer happens to have multiple processors, it can operate on several blocks in parallel. The use of blocks simplifies the organization

of parallel computing. It also helps to minimize the bottlenecks associated with communication between processors. This latter benefit also stems from the fact that $O(r^3)$ flops are performed on $O(r^2)$ data items.

Many of the algorithms that will be considered in this book can be organized into blocks, as we shall see. The public-domain linear algebra subroutine library LAPACK [1] uses block algorithms wherever it can.

### Fast Matrix Multiplication

If we multiply two $n \times n$ matrices together using the definition (1.1.12), $2n^3$ flops are required. In 1969 V. Strassen [68] amazed the computing world by presenting a method that can do the job in $O(n^s)$ flops, where $s = \log_2 7 \approx 2.81$. Since $2.81 < 3$, Strassen's method will be faster than conventional algorithms if $n$ is sufficiently large. Tests have shown that even for $n$ as small as 100 or so, Strassen's method can be faster. However, since 2.81 is only slightly less than 3, $n$ has to be made quite large before Strassen's method wins by a large margin. Accuracy is also an issue. Strassen's method has not made a great impact so far, but that could change in the future.

Even "faster" methods have been found. The current record holder, due to Coppersmith and Winograd, can multiply two $n \times n$ matrices in about $O(n^{2.376})$ flops. But there is a catch. When we write $O(n^{2.376})$, we mean that there is a constant $C$ such that the algorithm takes no more than $Cn^{2.376}$ flops. For this algorithm the constant $C$ is so large that it does not beat Strassen's method until $n$ is really enormous.

A good overview of fast matrix multiplication methods is given by Higham [41].

## 1.2 SYSTEMS OF LINEAR EQUATIONS

In the previous section we discussed the problem of multiplying a matrix $A$ times a vector $x$ to obtain a vector $b$. In scientific computations one is more likely to have to solve the inverse problem: Given $A$ (an $n \times n$ matrix) and $b$, solve for $x$. That is, find $x$ such that $Ax = b$. This is the problem of solving a system of $n$ linear equations in $n$ unknowns.

You have undoubtedly already had some experience solving systems of linear equations. We will begin this section by reminding you briefly of some of the basic theoretical facts. We will then look at several simple examples to remind you of how linear systems can arise in scientific problems.

## Nonsingularity and Uniqueness of Solutions

Consider a system of $n$ linear equations in $n$ unknowns

$$\begin{aligned} a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n &= b_1 \\ a_{21}x_1 + a_{22}x_2 + \cdots + a_{2n}x_n &= b_2 \\ &\vdots \\ a_{n1}x_1 + a_{n2}x_2 + \cdots + a_{nn}x_n &= b_n. \end{aligned} \tag{1.2.1}$$

The coefficients $a_{ij}$ and $b_i$ are given, and we wish to find $x_1, \ldots x_n$ that satisfy the equations. In most applications the coefficients are real numbers, and we seek a real solution. Therefore we will confine our attention to real systems. However, everything we will do can be carried over to the complex number field. (In some situations minor modifications are required. These will be covered in the exercises.)

Since it is tedious to write out (1.2.1) again and again, we generally prefer to write it as a single matrix equation

$$Ax = b, \tag{1.2.2}$$

where

$$A = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} \end{bmatrix} \quad x = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} \quad b = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_n \end{bmatrix}.$$

$A$ and $b$ are given, and we must solve for $x$. $A$ is a square matrix; it has $n$ rows and $n$ columns.

Equation (1.2.2) has a unique solution if and only if the matrix $A$ is nonsingular. Theorem 1.2.3 summarizes some of the simple characterizations of nonsingularity that we will use in this book.

First we recall some standard terminology. The $n \times n$ *identity matrix* is denoted by $I$. It is the unique matrix such that $AI = IA = A$ for all $A \in \mathbb{R}^{n\times n}$. The identity matrix has 1's on its main diagonal and 0's elsewhere. For example, the $3 \times 3$ identity matrix has the form

$$I = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}.$$

Given a matrix $A$, if there is a matrix $B$ such that $AB = BA = I$, then $B$ is called the *inverse* of $A$ and denoted $A^{-1}$. Not every matrix has an inverse.

**Theorem 1.2.3** *Let $A$ be a square matrix. The following six conditions are equivalent; that is, if any one holds, they all hold.*

*(a) $A^{-1}$ exists.*

*(b) There is no nonzero $y$ such that $Ay = 0$.*

*(c) The columns of $A$ are linearly independent.*

*(d) The rows of $A$ are linearly independent.*

*(e)* $\det(A) \neq 0$.

*(f) Given any vector $b$, there is exactly one vector $x$ such that $Ax = b$.*

*(In condition (b), the symbol 0 stands for the vector whose entries are all zero. In condition (e), the symbol 0 stands for the real number 0.* $\det(A)$ *denotes the determinant of $A$.)*

For a proof of Theorem 1.2.3 see any elementary linear algebra text. If the conditions of Theorem 1.2.3 hold, $A$ is said to be *nonsingular* or *invertible*. If the conditions do not hold, $A$ is said to be *singular* or *noninvertible*. In this case (1.2.2) has either no solution or infinitely many solutions. In this chapter we will focus on the nonsingular case.

If $A$ is nonsingular, the unique solution of (1.2.2) can be obtained in principle by multiplying both sides by $A^{-1}$: From $Ax = b$ we obtain $A^{-1}Ax = A^{-1}b$, and since $A^{-1}A = I$, the identity matrix, we obtain $x = A^{-1}b$. This equation solves the problem completely in theory, but the method of solution that it suggests—first calculate $A^{-1}$, then multiply $A^{-1}$ by $b$ to obtain $x$—is usually a bad idea. As we shall see, it is generally more efficient to solve $Ax = b$ directly, without calculating $A^{-1}$. On most large problems the savings in computation and storage achieved by avoiding the use of $A^{-1}$ are truly spectacular.

Situations in which the inverse really needs to be calculated are quite rare. This does not imply that the inverse is unimportant; it is an extremely useful theoretical tool.

**Exercise 1.2.4** Prove that if $A^{-1}$ exists, then there can be no nonzero $y$ for which $Ay = 0$. □

**Exercise 1.2.5** Prove that if $A^{-1}$ exists, then $\det(A) \neq 0$. □

We now move on to some examples.

### Electrical Circuits

**Example 1.2.6** Consider the electrical circuit shown in Figure 1.1. Suppose the circuit is in an equilibrium state; all of the voltages and currents are constant. The four unknown nodal voltages $x_1, \ldots, x_4$ can be determined as follows. At each of the four nodes, the sum of the currents away from the node must be zero (Kirchhoff's current law). This gives us an equation for each node. In each of these equations the currents can be expressed in terms of voltages using Ohm's law, which states that the voltage drop (in volts) is equal to the current (in amperes) times the resistance (in ohms). For example, suppose the current from node 3 to node 4 through the 5 Ω resistor is $I$. Then by Ohm's law, $x_3 - x_4 = 5I$, so $I = .2(x_3 - x_4)$. Treating the

other two currents flowing from node 3 in the same way, and applying Kirchhoff's current law, we get the equation

$$.2(x_3 - x_4) + 1(x_3 - x_1) + .5(x_3 - 6) = 0$$

or

$$-x_1 + 1.7x_3 - .2x_4 = 3.$$

Applying the same procedure to nodes 1, 2, and 4, we obtain a system of four linear equations in four unknowns:

$$\begin{array}{rcrcrcrcl} & 2x_1 & - & x_2 & - & x_3 & & = & 0 \\ - & x_1 & + & 1.5x_2 & & & - \; .5x_4 & = & 0 \\ - & x_1 & & & + & 1.7x_3 & - \; .2x_4 & = & 3 \\ & & - & .5x_2 & - & .2x_3 & + \; 1.7x_4 & = & 0 \end{array},$$

which can be written as a single matrix equation

$$\begin{bmatrix} 2 & -1 & -1 & 0 \\ -1 & 1.5 & 0 & -.5 \\ -1 & 0 & 1.7 & -.2 \\ 0 & -.5 & -.2 & 1.7 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 3 \\ 0 \end{bmatrix}.$$

Though we will not prove it here, the coefficient matrix is nonsingular, so the system has exactly one solution. Solving it using MATLAB, we find that

$$x = \begin{bmatrix} 3.0638 \\ 2.4255 \\ 3.7021 \\ 1.1489 \end{bmatrix}.$$

Thus, for example, the voltage at node 3 is 3.7021 volts. These are not the exact answers; they are rounded off to four decimal places. Given the nodal voltages,

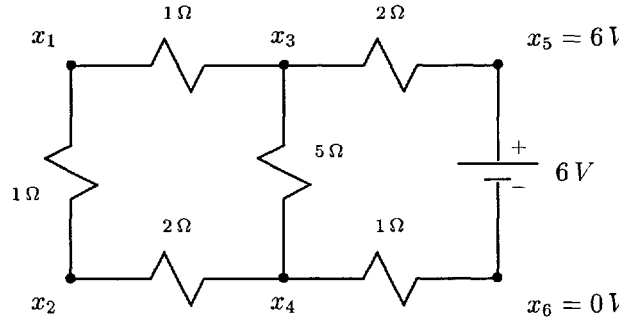



***Fig. 1.1*** Solve for the nodal voltages.

we can easily calculate the current through any of the resistors by Ohm's law. For example, the current flowing from node 3 to node 4 is $.2(x_3 - x_4) = 0.5106$ amperes. □

**Exercise 1.2.7** Verify the correctness of the equations in Example 1.2.6. Use MATLAB (or some other means) to compute the solution. If you are unfamiliar with MATLAB, you can use the MATLAB demo (Start MATLAB and type `demo`) to find out how to enter matrices. Once you have entered the matrix `A` and the vector `b`, you can type `x = A\b` to solve for $x$. A transcript of your whole session (which you can later edit, print out, and turn in to your instructor) can be made by using the command `diary on`. For more information about the `diary` command type `help diary`. □

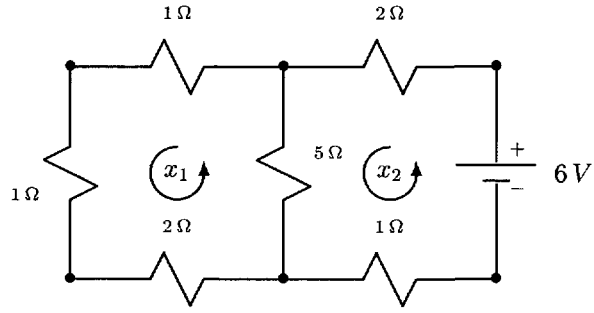



***Fig. 1.2*** Solve for the loop currents.

**Example 1.2.8** Another way to analyze a planar electrical circuit is to solve for loop currents instead of nodal voltages. Figure 1.2 shows the same circuit as before, but now we have associated currents $x_1$ and $x_2$ with the two loops. (These are clearly not the same $x_i$ as in the previous figure.) For the resistors that lie on the boundary of the circuit, the loop current is the actual current flowing through the resistor, but the current flowing through the 5 Ω resistor in the middle is the difference $x_2 - x_1$. An equation for each loop can be obtained by applying the principle that the sum of the voltage drops around the loop must be zero (Kirchhoff's voltage law). The voltage drop across each resistor can be expressed in terms of the loop currents by applying Ohm's law. For example, the voltage drop across the 5 Ω resistor, from top to bottom, is $5(x_2 - x_1)$ volts. Summing the voltage drops across the four resistors in loop 1, we obtain

$$5(x_1 - x_2) + 1x_1 + 1x_1 + 2x_1 = 0.$$

Similarly, in loop 2,

$$5(x_2 - x_1) + 1x_2 - 6 + 2x_2 = 0.$$

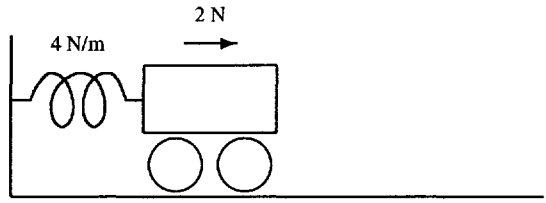



***Fig. 1.3*** Single cart and spring

Rearranging these equations, we obtain the $2 \times 2$ system

$$\begin{bmatrix} 9 & -5 \\ -5 & 8 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 0 \\ 6 \end{bmatrix}.$$

Solving these equations by hand, we find that $x_1 = 30/47 = 0.6383$ amperes and $x_2 = 54/47 = 1.1489$ amperes. Thus, for example, the current through the $5\,\Omega$ resistor, from top to bottom, is $x_2 - x_1 = .5106$ amperes, and the voltage drop is 2.5532 volts. These results are in agreement with those of Example 1.2.6. □

**Exercise 1.2.9** Check that the equations in Example 1.2.8 are correct. Check that the coefficient matrix is nonsingular. Solve the system by hand, by MATLAB, or by some other means. □

It is easy to imagine much larger circuits with many loops. See, for example, Exercise 1.2.19. Then imagine something much larger. If a circuit has, say, 100 loops, then it will have 100 equations in 100 unknowns.

### Simple Mass-Spring Systems

In Figure 1.3 a steady force of 2 newtons is applied to a cart, pushing it to the right and stretching the spring, which is a linear spring with a spring constant (stiffness) 4 newtons/meter. How far will the cart move before stopping at a new equilibrium position? Here we are not studying the dynamics, that is, how the cart gets to its new equilibrium. For that we would need to know the mass of the cart and the frictional forces in the system. Since we are asking only for the new equilibrium position, it suffices to know the stiffness of the spring.

The new equilibrium will be at the point at which the rightward force of 2 newtons is exactly balanced by the leftward force applied by the spring. In other words, the equilibrium position is the one at which the sum of the forces on the cart is zero. Let $x$ denote the (yet unknown) amount by which the cart moves to the right. Then the restoring force of the spring is $-4$ newtons/meter $\times$ $x$ meters $= -4x$ newtons. It is

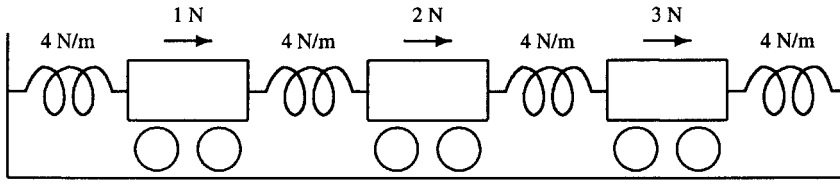



***Fig. 1.4*** System of three carts

negative because it pulls the cart leftward. The equilibrium occurs when $-4x+2 = 0$. Solving this system of one equation in one unknown, we find that $x = 0.5$ meter.

**Example 1.2.10** Now suppose we have three masses attached by springs as shown in Figure 1.4. Let $x_1$, $x_2$, and $x_3$ denote the amount by which carts 1, 2, and 3, respectively, move when the forces are applied. For each cart the new equilibrium position is that point at which the sum of the forces on the cart is zero. Consider the second cart, for example. An external force of two newtons is applied, and there is the leftward force of the spring to the left, and the rightward force of the spring to the right. The amount by which the spring on the left is stretched is $x_2 - x_1$ meters. It therefore exerts a force $-4$ newtons/meter $\times$ $(x_2 - x_1)$ meters $= -4(x_2 - x_1)$ newtons on the second cart. Similarly the spring on the right applies a force of $+4(x_3 - x_2)$ newtons. Thus the equilibrium equation for the second cart is

$$-4(x_2 - x_1) + 4(x_3 - x_2) + 2 = 0$$

or

$$-4x_1 + 8x_2 - 4x_3 = 2.$$

Similar equations apply to carts 1 and 3. Thus we obtain a system of three linear equations in three unknowns, which we can write as a matrix equation

$$\begin{bmatrix} 8 & -4 & 0 \\ -4 & 8 & -4 \\ 0 & -4 & 8 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}.$$

Entering the matrix $A$ and vector $b$ into MATLAB, and using the command `x = A\b` (or simply solving the system by hand) we find that

$$x = \begin{bmatrix} 0.625 \\ 1.000 \\ 0.875 \end{bmatrix}.$$

Thus the first cart is displaced to the right by a distance of 0.625 meters, for example.

The coefficient matrix $A$ is called a *stiffness matrix*, because the values of its nonzero entries are determined by the stiffnesses of the springs. □

**Exercise 1.2.11** Check that the equations in Example 1.2.10 are correct. Check that the coefficient matrix of the system is nonsingular. Solve the system by hand, by MATLAB, or by some other means. □

It is easy to imagine more complex examples. If we have $n$ carts in a line, we get a system of $n$ equations in $n$ unknowns. See Exercise 1.2.20. We can also consider problems in which masses are free to move in two or three dimensions and are connected by a network of springs.

### Numerical (Approximate) Solution of Differential Equations

Many physical phenomena can be modeled by differential equations. We shall consider one example without going into too many details.

**Example 1.2.12** Consider a differential equation

$$-u''(x) + cu'(x) + du(x) = f(x) \qquad 0 < x < 1 \tag{1.2.13}$$

with boundary conditions

$$u(0) = 0, \quad u(1) = 0. \tag{1.2.14}$$

The problem is to solve for the function $u$, given the constants $c$ and $d$ and the function $f$. For example, $u(x)$ could represent the unknown concentration of some chemical pollutant at distance $x$ from the end of a pipe.[2] Depending on the function $f$, it may or may not be within our power to solve this boundary value problem exactly. If not, we can solve it approximately by the finite difference method, as follows.

Pick a (possibly large) integer $m$, and subdivide the interval $[0, 1]$ into $m$ equal subintervals of length $h = 1/m$. The subdivision points of the intervals are $x_j = jh$, $j = 0, \ldots, m$. These points constitute our computational grid. The finite difference technique will produce approximations to $u(x_1), \ldots, u(x_{m-1})$. Since (1.2.13) holds at each grid point, we have

$$-u''(x_i) + cu'(x_i) + du(x_i) = f(x_i) \qquad i = 1, \ldots, m-1.$$

If $h$ is small, good approximations for the first and second derivatives are

$$u'(x_i) \approx \frac{u(x_i + h) - u(x_i - h)}{2h} = \frac{u(x_{i+1}) - u(x_{i-1})}{2h}$$

and

$$u''(x_i) \approx \frac{u(x_{i+1}) - 2u(x_i) + u(x_{i-1})}{h^2}.$$

(See Exercise 1.2.21.) Substituting these approximations for the derivatives into the differential equation, we obtain, for $i = 1, \ldots, m-1$,

$$\frac{-u(x_{i+1}) + 2u(x_i) - u(x_{i-1})}{h^2} + c\left(\frac{u(x_{i+1}) - u(x_{i-1})}{2h}\right) + du(x_i) \approx f(x_i).$$

[2]The term $-u''(x)$ is a diffusion term, $cu'(x)$ is a convection term, $du(x)$ is a decay term, and $f(x)$ is a source term.

We now approximate this by a system of difference equations

$$\frac{-u_{i+1} + 2u_i - u_{i-1}}{h^2} + c\left(\frac{u_{i+1} - u_{i-1}}{2h}\right) + du_i = f_i, \tag{1.2.15}$$

$i = 1, \ldots, m-1$. Here we have replaced the approximation symbol by an equal sign and $u(x_i)$ by the symbol $u_i$, which (hopefully) is an approximation of $u(x_i)$. We have also introduced the symbol $f_i$ as an abbreviation for $f(x_i)$. This is a system of $m-1$ linear equations in the unknowns $u_0, u_1, \ldots, u_m$. Applying the boundary conditions (1.2.14), we can take $u_0 = 0$ and $u_m = 0$, leaving only $m-1$ unknowns $u_1, \ldots, u_{m-1}$.

Suppose, for example, $m = 6$ and $h = 1/6$. Then (1.2.15) is a system of five equations in five unknowns, which can be written as the single matrix equation

$$\begin{bmatrix} 72+d & -36+3c & 0 & 0 & 0 \\ -36-3c & 72+d & -36+3c & 0 & 0 \\ 0 & -36-3c & 72+d & -36+3c & 0 \\ 0 & 0 & -36-3c & 72+d & -36+3c \\ 0 & 0 & 0 & -36-3c & 72+d \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \\ u_3 \\ u_4 \\ u_5 \end{bmatrix} = \begin{bmatrix} f_1 \\ f_2 \\ f_3 \\ f_4 \\ f_5 \end{bmatrix}.$$

Given specific $c$, $d$, and $f$, we can solve this system of equations for $u_1, \ldots, u_5$. Since the difference equations mimic the differential equation, we expect that $u_1, \ldots, u_5$ will approximate the true solution of the boundary value problem at the points $x_1, \ldots x_5$.

Of course, we do not expect a very good approximation when we take only $m = 6$. To get a good approximation, we should take $m$ much larger, which results in a much larger system of equations to solve. □

**Exercise 1.2.16** Write the system of equations from Example 1.2.12 as a matrix equation for (a) $m = 8$, (b) $m = 20$. □

More complicated systems of difference equations arising from partial differential equations are discussed in Section 7.1.

### Additional Exercises

**Exercise 1.2.17** Consider the electrical circuit in Figure 1.5.

(a) Write down a linear system $Ax = b$ with seven equations for the seven unknown nodal voltages.

(b) Using MATLAB, for example, solve the system to find the nodal voltages. Calculate the residual $r = b - A\hat{x}$, where $\hat{x}$ denotes your computed solution. In theory $r$ should be zero. In practice you will get a tiny but nonzero residual because of roundoff errors in your computation. Use the `diary` command to make a transcript of your session that you can turn in to your instructor.

□

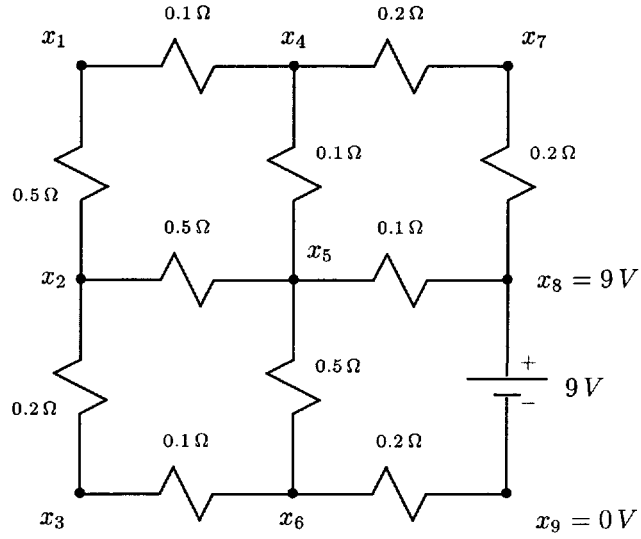



***Fig. 1.5*** Electric circuit with nodal voltages

**Exercise 1.2.18** The circuit in Figure 1.6 is the same as for the previous problem, but now let us focus on the loop currents.

(a) Write down a linear system $Ax = b$ of four equations for the four unknown loop currents.

(b) Solve the system for $x$. Calculate the residual $r = b - A\hat{x}$, where $\hat{x}$ denotes your computed solution.

(c) Using Ohm's law and the loop currents that you just calculated, find the voltage drops from the node labeled $n_1$ to the nodes labeled $n_2$ and $n_3$. Are these in agreement with your solution to Exercise 1.2.17?

(d) Using Ohm's law and the voltages calculated in Exercise 1.2.17, find the current through the resistor labeled $R_1$. Is this in agreement with your loop current calculation?

□

**Exercise 1.2.19** In the circuit in Figure 1.7 all of the resistances are 1 Ω.

(a) Write down a linear system $Ax = b$ that you can solve for the loop currents.

(b) Solve the system for $x$.

□

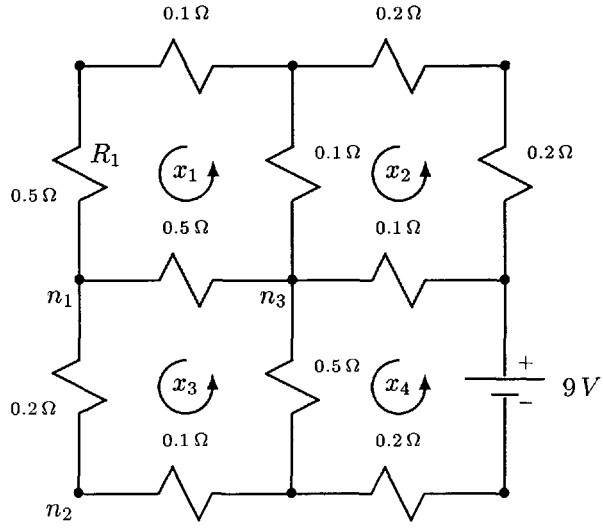



***Fig. 1.6*** Electrical circuit with loop currents

**Exercise 1.2.20** Consider a system of $n$ carts connected by springs, as shown in Figure 1.8. The $i$th spring has a stiffness of $k_i$ newtons/meter. Suppose that the carts are subjected to steady forces of $f_1, f_2, \ldots, f_n$ newtons, respectively, causing displacements of $x_1, x_2, \ldots, x_n$ meters, respectively.

(a) Write down a system of $n$ linear equations $Ax = b$ that could be solved for $x_1, \ldots, x_n$. Notice that if $n$ is at all large, the vast majority of the entries of $A$ will be zeros. Matrices with this property are called *sparse*. Since all of the nonzeros are confined to a narrow band around the main diagonal, $A$ is also called *banded*. In particular, the nonzeros are confined to three diagonals, so $A$ is *tridiagonal*.

(b) Compute the solution in the case $n = 20$, $k_i = 1$ newton/meter for all $i$, and $f_i = 0$ except for $f_5 = 1$ newton and $f_{16} = -1$ newton. (Type `help toeplitz` to learn an easy way to enter the coefficient matrix in MATLAB.)

□

**Exercise 1.2.21** Recall the definition of the derivative from elementary calculus:

$$u'(x) = \lim_{h \to 0} \frac{u(x+h) - u(x)}{h}.$$

(a) Show that if $h$ is a sufficiently small positive number, then both

$$\frac{u(x+h) - u(x)}{h} \quad \text{and} \quad \frac{u(x) - u(x-h)}{h}$$

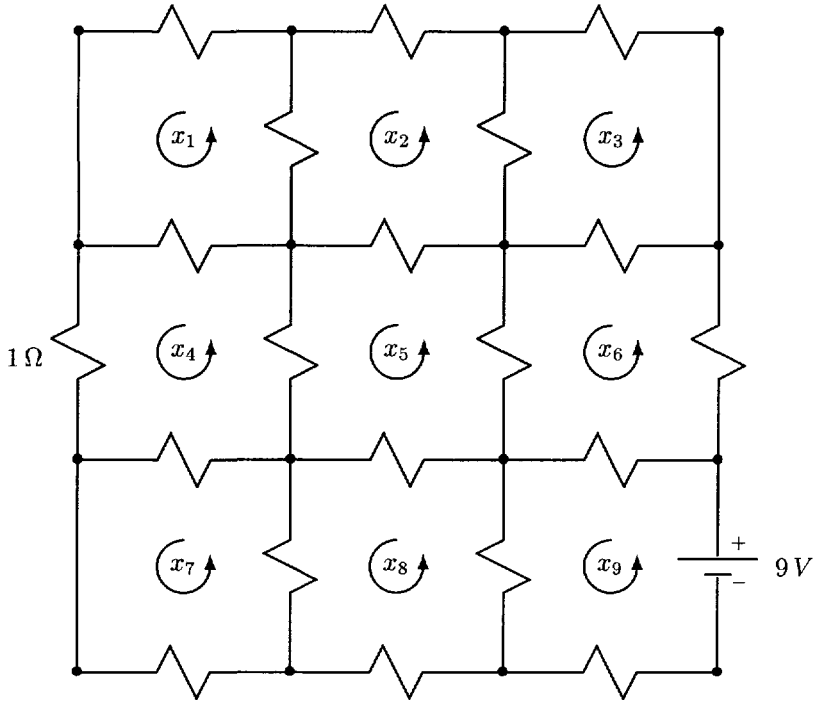



***Fig. 1.7*** Calculate the loop currents.

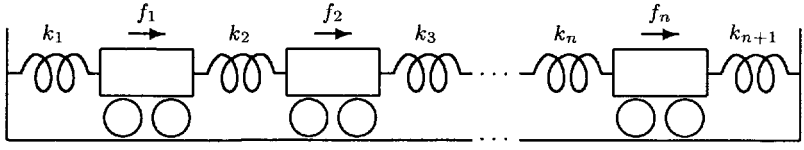



***Fig. 1.8*** System of $n$ masses

are good approximations of $u'(x)$.

(b) Take the average of the two estimates from part (a) to obtain the estimate

$$u'(x) \approx \frac{u(x+h) - u(x-h)}{2h}.$$

Draw a picture (the graph of $u$ and a few straight lines) that shows that this estimate is likely to be a better estimate of $u'(x)$ than either of the estimates from part (a) are.

(c) Apply the estimate from part (b) to $u''(x)$ with $h$ replaced by $h/2$ to obtain

$$u''(x) \approx \frac{u'(x+h/2) - u'(x-h/2)}{h}.$$

Now approximate $u'(x+h/2)$ and $u'(x-h/2)$ using the estimate from part (b), again with $h$ replaced by $h/2$, to obtain

$$u''(x) \approx \frac{u(x+h) - 2u(x) + u(x-h)}{h^2}.$$

□

## 1.3 TRIANGULAR SYSTEMS

A linear system whose coefficient matrix is triangular is particularly easy to solve. It is a common practice to reduce general systems to a triangular form, which can then be solved inexpensively. For this reason we will study triangular systems in detail.

A matrix $G = (g_{ij})$ is *lower triangular* if $g_{ij} = 0$ whenever $i < j$. Thus a lower-triangular matrix has the form

$$G = \begin{bmatrix} g_{11} & 0 & 0 & \cdots & 0 \\ g_{21} & g_{22} & 0 & \cdots & 0 \\ g_{31} & g_{32} & g_{33} & \ddots & \vdots \\ \vdots & \vdots & \vdots & \ddots & 0 \\ g_{n1} & g_{n2} & g_{n3} & \cdots & g_{nn} \end{bmatrix}.$$

Similarly, an *upper triangular* matrix is one for which $g_{ij} = 0$ whenever $i > j$. A *triangular* matrix is one that is either upper or lower triangular.

**Theorem 1.3.1** *Let $G$ be a triangular matrix. Then $G$ is nonsingular if and only if $g_{ii} \neq 0$ for $i = 1, \ldots, n$.*

**Proof.** Recall from elementary linear algebra that if $G$ is triangular, then $\det(G) = g_{11}g_{22}\cdots g_{nn}$. Thus $\det(G) \neq 0$ if and only if $g_{ii} \neq 0$ for $i = 1, \ldots, n$. See Exercises 1.3.23 and 1.3.24. □

## Lower-Triangular Systems

Consider the system

$$Gy = b, \tag{1.3.2}$$

where $G$ is a nonsingular, lower-triangular matrix. It is easy to see how to solve this system if we write it out in detail:

$$\begin{array}{lllllllll}
g_{11}y_1 & & & & & & & = & b_1 \\
g_{21}y_1 & + & g_{22}y_2 & & & & & = & b_2 \\
g_{31}y_1 & + & g_{32}y_2 & + & g_{33}y_3 & & & = & b_3 \\
 & & \vdots & & & & & \vdots & \\
g_{n1}y_1 & + & g_{n2}y_2 & + & g_{n3}y_3 & + \cdots + & g_{nn}y_n & = & b_n.
\end{array}$$

The first equation involves only the unknown $y_1$, the second involves only $y_1$ and $y_2$, and so on. We can solve the first equation for $y_1$:

$$y_1 = b_1/g_{11}.$$

The assumption that $G$ is nonsingular ensures that $g_{11} \neq 0$. Now that we have $y_1$, we can substitute its value into the second equation and solve that equation for $y_2$:

$$y_2 = (b_2 - g_{21}y_1)/g_{22}.$$

Since $G$ is nonsingular, $g_{22} \neq 0$. Now that we know $y_2$, we can use the third equation to solve for $y_3$, and so on. In general, once we have $y_1, y_2, \ldots, y_{i-1}$, we can solve for $y_i$, using the $i$th equation:

$$y_i = \frac{b_i - g_{i1}y_1 - g_{i2}y_2 - \cdots - g_{i,i-1}y_{i-1}}{g_{ii}},$$

which can be expressed more succinctly using sigma notation:

$$y_i = g_{ii}^{-1}\left(b_i - \sum_{j=1}^{i-1} g_{ij}y_j\right). \tag{1.3.3}$$

Since $G$ is nonsingular, $g_{ii} \neq 0$.

This algorithm for solving a lower-triangular system is called *forward substitution* or *forward elimination*. This is the first of two versions we will consider. It is called *row-oriented* forward substitution because it accesses $G$ by rows; the $i$th row is used at the $i$th step. It is also called the *inner-product* form of forward substitution because the sum $\sum_{j=1}^{i-1} g_{ij}y_j$ can be regarded as an inner or dot product.

Equation (1.3.3) describes the algorithm completely; it even describes the first step ($y_1 = b_1/g_{11}$), if we agree, as we shall throughout the book, that whenever the lower limit of a sum is greater than the upper limit, the sum is zero. Thus

$$y_1 = \frac{b_1 - \sum_{j=1}^{0} g_{1j}y_j}{g_{11}} = \frac{b_1 - 0}{g_{11}} = \frac{b_1}{g_{11}}.$$

**Exercise 1.3.4** Use pencil and paper to solve the system

$$\begin{bmatrix} 2 & 0 & 0 & 0 \\ -1 & 2 & 0 & 0 \\ 3 & 1 & -1 & 0 \\ 4 & 1 & -3 & 3 \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \\ y_3 \\ y_4 \end{bmatrix} = \begin{bmatrix} 2 \\ 3 \\ 2 \\ 9 \end{bmatrix}$$

by forward substitution. You can easily check your answer by substituting it back into the equations. This is a simple means of checking you work that you will be able to use on many of the hand computation exercises that you will be asked to perform in this chapter. □

It would be easy to write a computer program for forward elimination. Before we write down the algorithm, notice that $b_1$ is only used in calculating $y_1$, $b_2$ is only used in calculating $y_2$, and so on. In general, once we have calculated $y_i$, we no longer need $b_i$. It is therefore usual for a computer program to store $y$ over $b$. Thus we have a single array that contains $b$ before the program is executed and $y$ afterward. The algorithm looks like this:

$$\begin{array}{l} \text{for } i = 1, \ldots, n \\ \quad \left[ \begin{array}{l} \text{for } j = 1, \ldots, i-1 \text{ (not executed when } i = 1) \\ \quad \left[ \; b_i \leftarrow b_i - g_{ij} b_j \right. \\ \text{if } g_{ii} = 0, \text{ set error flag, exit} \\ b_i \leftarrow b_i / g_{ii} \end{array} \right. \end{array} \tag{1.3.5}$$

There are no references to $y$, since it is stored in the array named $b$. The check of $g_{ii}$ is included to make the program foolproof. There is nothing to guarantee that the program will not at some time be given (accidentally or otherwise) a singular coefficient matrix. The program needs to be able to respond appropriately to this situation. It is a good practice to check before each division that the divisor is not zero. In most linear algebra algorithms these checks do not contribute significantly to the time it takes to run the program, because the division operation is executed relatively infrequently.

To get an idea of the execution time of forward substitution, let us count the floating-point operations (flops). In the inner loop of (1.3.5), two flops are executed. These flops are performed $i - 1$ times on the $i$th time through the outer loop. The outer loop is performed $n$ times, so the total number of flops performed in the $j$ loop is $2 \times (0 + 1 + 2 + \cdots + n - 1) = 2\sum_{i=1}^{n}(i-1)$. Calculating this sum by a well-known trick (see Exercises 1.3.25, 1.3.26, and 1.3.28), we get $n(n-1)$, which is approximated well by the simpler expression $n^2$. These considerations are summarized by the equations

$$\sum_{i=1}^{n} \sum_{j=1}^{i-1} 2 = 2 \sum_{i=1}^{n} (i-1) = n(n-1) \approx n^2.$$

Looking at the operations that are performed outside the $j$ loop, we see that $g_{ii}$ is compared with zero $n$ times, and there are $n$ divisions. Regardless of what each of

these operations costs, the total cost of doing all of them is proportional to $n$, not $n^2$, and will therefore be insignificant if $n$ is at all large. Making this assumption, we ignore the lesser costs and state simply that the cost of doing forward substitution is $n^2$ flops.

This figure gives us valuable qualitative information. We can expect that each time we double $n$, the execution time of forward substitution will be multiplied by approximately four.

### Exploiting Leading Zeros in Forward Substitution

Significant savings can be achieved if some of the leading entries of $b$ are zero. This observation will prove important when we study banded matrix computations in Section 1.5. First suppose $b_1 = 0$. Then obviously $y_1 = 0$ as well, and we do not need to make the computer do the computation $y_1 = b_1/g_{11}$. In addition, all subsequent computations involving $y_1$ can be skipped. Now suppose that $b_2 = 0$ also. Then $y_2 = b_2/g_{22} = 0$. There is no need for the computer to carry out this computation or any subsequent computation involving $y_2$. In general, if $b_1 = b_2 = \cdots b_k = 0$, then $y_1 = y_2 = \cdots = y_k = 0$, and we can skip all of the computations involving $y_1$, $\ldots y_k$. Thus (1.3.3) becomes

$$y_i = \frac{b_i - \sum_{j=k+1}^{n} g_{ij}y_j}{g_{ii}} \qquad i = k+1, \ldots, n. \tag{1.3.6}$$

Notice that the sum begins at $j = k + 1$.

It is enlightening to look at this from the point of view of partitioned matrices. If $b_1 = b_2 = \cdots = b_k = 0$, we can write

$$b = \begin{bmatrix} 0 \\ \hat{b}_2 \end{bmatrix} \begin{matrix} k \\ j \end{matrix},$$

where $j = n - k$. Partitioning $G$ and $y$ also, we have

$$G = \begin{matrix} \\ k \\ j \end{matrix} \begin{matrix} \begin{matrix} k & j \end{matrix} \\ \begin{bmatrix} G_{11} & 0 \\ G_{21} & G_{22} \end{bmatrix} \end{matrix} \qquad y = \begin{bmatrix} \hat{y}_1 \\ \hat{y}_2 \end{bmatrix} \begin{matrix} k \\ j \end{matrix},$$

where $G_{11}$ and $G_{22}$ are lower triangular. The equation $Gy = b$ becomes

$$\begin{bmatrix} G_{11} & 0 \\ G_{21} & G_{22} \end{bmatrix} \begin{bmatrix} \hat{y}_1 \\ \hat{y}_2 \end{bmatrix} = \begin{bmatrix} 0 \\ \hat{b}_2 \end{bmatrix}$$

or

$$\begin{aligned} G_{11}\hat{y}_1 &= 0 \\ G_{21}\hat{y}_1 + G_{22}\hat{y}_2 &= \hat{b}_2. \end{aligned}$$

Since $G_{11}$ is nonsingular (why?), the first equation implies $\hat{y}_1 = 0$. The second equation then reduces to

$$G_{22}\hat{y}_2 = \hat{b}_2.$$

Thus we only have to solve this $(n-k) \times (n-k)$ lower-triangular system, which is exactly what (1.3.6) does. $G_{11}$ and $G_{21}$ are not used, because they interact only with the unknowns $y_1, \ldots, y_k$, (i.e. $\hat{y}_1$). Since the system now being solved is of order $n-k$, the cost is now $(n-k)^2$ flops.

**Exercise 1.3.7** Write a modified version of Algorithm (1.3.5) that checks for leading zeros in $b$ and takes appropriate action. □

### Column-Oriented Forward Substitution

We now derive a column-oriented version of forward substitution. Partition the system $Gy = b$ as follows:

$$\begin{bmatrix} g_{11} & 0 \\ \hat{h} & \hat{G} \end{bmatrix} \begin{bmatrix} y_1 \\ \hat{y} \end{bmatrix} = \begin{bmatrix} b_1 \\ \hat{b} \end{bmatrix}. \tag{1.3.8}$$

$\hat{h}$, $\hat{y}$, and $\hat{b}$ are vectors of length $n-1$, and $\hat{G}$ is an $(n-1) \times (n-1)$ lower-triangular matrix. The partitioned system can be written as

$$\begin{array}{rcrcl} g_{11}y_1 & & & = & b_1 \\ \hat{h}y_1 & + & \hat{G}\hat{y} & = & \hat{b}. \end{array}$$

This leads to the following algorithm:

$$\begin{array}{l} y_1 = b_1/g_{11} \\ \tilde{b} = \hat{b} - \hat{h}y_1 \\ \text{solve } \hat{G}\hat{y} = \tilde{b} \text{ for } \hat{y}. \end{array} \tag{1.3.9}$$

This algorithm reduces the problem of solving an $n \times n$ triangular system to that of solving the $(n-1) \times (n-1)$ system $\hat{G}\hat{y} = \tilde{b}$. This smaller problem can be reduced (by the same algorithm) to a problem of order $n-2$, which can in turn be reduced to a problem of order $n-3$, and so on. Eventually we get to the $1 \times 1$ problem $g_{nn}y_n = \tilde{b}_n$, which has the solution $y_n = \tilde{b}_n/g_{nn}$.

If you are a student of mathematics, this algorithm should remind you of proof by induction. If, on the other hand, you are a student of computer science, you might think of recursion, which is the computer science analogue of mathematical induction. Recall that a *recursive* procedure is one that calls itself. If you like to program in a computer language that supports recursion (and most modern languages do), you might enjoy writing a recursive procedure that implements (1.3.9). The procedure would perform steps one and two of (1.3.9) and then call itself to solve the reduced problem. All variables named $b$, $\hat{b}$, $\tilde{b}$, $y$, and so on, can be stored in a single array, which will contain $b$ before execution and $y$ afterward.

Although it is fun to write recursive programs, this algorithm can also be coded nonrecursively without difficulty. We will write down a nonrecursive formulation of the algorithm, but first it is worthwhile to work through one or two examples by hand.

**Example 1.3.10** Let us use the column-oriented version of forward substitution to solve the lower-triangular system

$$\begin{bmatrix} 5 & 0 & 0 \\ 2 & -4 & 0 \\ 1 & 2 & 3 \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \\ y_3 \end{bmatrix} = \begin{bmatrix} 15 \\ -2 \\ 10 \end{bmatrix}.$$

First we calculate $y_1 = b_1/g_{11} = 15/5 = 3$. Then

$$\tilde{b} = \hat{b} - \hat{h}y_1 = \begin{bmatrix} -2 \\ 10 \end{bmatrix} - \begin{bmatrix} 2 \\ 1 \end{bmatrix} 3 = \begin{bmatrix} -8 \\ 7 \end{bmatrix}.$$

Now we have to solve $\hat{G}\hat{y} = \tilde{b}$:

$$\begin{bmatrix} -4 & 0 \\ 2 & 3 \end{bmatrix} \begin{bmatrix} y_2 \\ y_3 \end{bmatrix} = \begin{bmatrix} -8 \\ 7 \end{bmatrix}.$$

We achieve this by repeating the algorithm: $y_2 = -8/-4 = 2$,

$$\tilde{\tilde{b}} = \hat{\tilde{b}} - \hat{\tilde{h}}y_2 = [\ 7\ ] - [\ 2\ ]\, 2 = [\ 3\ ],$$

$[\ 3\ ]\, y_3 = [\ 3\ ]$, and $y_3 = 3/3 = 1$. Thus

$$y = \begin{bmatrix} 3 \\ 2 \\ 1 \end{bmatrix}.$$

You can check that if you multiply $G$ by $y$, you get the correct right hand side $b$. □

**Exercise 1.3.11** Use column-oriented forward substitution to solve the system from Exercise 1.3.4. □

**Exercise 1.3.12** Write a nonrecursive algorithm in the spirit of (1.3.5) that implements column-oriented forward substitution. Use a single array that contains $b$ initially, stores intermediate results (e.g. $\hat{b}$, $\tilde{b}$) during the computation, and contains $y$ at the end. □

Your solution to Exercise 1.3.12 should look something like this:

```
for j = 1, ..., n
    [ if g_jj = 0, set error flag (G is singular), exit
      b_j ← b_j/g_jj  (this is y_j)
      for i = j + 1, ..., n  (not executed when j = n)
          [ b_i ← b_i − g_ij b_j
```

(1.3.13)

Notice that (1.3.13) accesses $G$ by columns, as anticipated: on the $j$th step, the $j$th column is used. Each time through the outer loop, the size of the problem is reduced by one. On the last time through, the computation $b_n \leftarrow b_n/g_{nn}$ (giving $y_n$) is performed, and the inner loop is skipped.

**Exercise 1.3.14**

(a) Count the operations in (1.3.13). Notice that the flop count is identical to that of the row-oriented algorithm (1.3.5).

(b) Convince yourself that the row- and column-oriented versions of forward substitution carry out exactly the same operations but not in the same order.

□

Like the row-oriented version, the column-oriented version can be modified to take advantage of any leading zeros that may occur in the vector $b$. On the $j$th time through the outer loop in (1.3.13), if $b_j = 0$, then no changes are made in $b$. Thus the $j$th step can be skipped whenever $b_j = 0$. (This is true regardless of whether or not $b_j$ is a leading zero. However, once a nonzero $b_j$ has been encountered, all subsequent $b_j$'s will not be the originals; they will have been altered on a previous step. Therefore they are not likely to be zero.)

Which of the two versions of forward substitution is superior? The answer depends on how $G$ is stored and accessed. This, in turn, depends on the programmer's choice of data structures and programming language and on the architecture of the computer.

### Upper-Triangular Systems

As you might expect, upper-triangular systems can be solved in much the same way as lower-triangular systems. Consider the system $Ux = y$, where $U$ is upper triangular. Writing out the system in detail we get

$$\begin{array}{ccccccccccc}
u_{11}x_1 & + & u_{12}x_2 & + & \cdots & + & u_{1,n-1}x_{n-1} & + & u_{1,n}x_n & = & y_1 \\
 & & u_{22}x_2 & + & \cdots & + & u_{2,n-1}x_{n-1} & + & u_{2n}x_n & = & y_2 \\
 & & & & & & \vdots & & & & \vdots \\
 & & & & & & u_{n-1,n-1}x_{n-1} & + & u_{n-1,n}x_n & = & y_{n-1} \\
 & & & & & & & & u_{nn}x_n & = & y_n.
\end{array}$$

It is clear that we should solve the system from bottom to top. The $n$th equation can be solved for $x_n$, then the $(n-1)$st equation can be solved for $x_{n-1}$, and so on. The process is called *back substitution*, and it has row- and column-oriented versions. The cost of back substitution is obviously the same as that of forward substitution, about $n^2$ flops.

**Exercise 1.3.15** Develop the row-oriented version of back substitution. Write pseudocode in the spirit of (1.3.5) and (1.3.13). □

**Exercise 1.3.16** Develop the column-oriented version of back substitution Write pseudocode in the spirit of (1.3.5) and (1.3.13). □

**Exercise 1.3.17** Solve the upper-triangular system

$$\begin{bmatrix} 3 & 2 & 1 & 0 \\ 0 & 1 & 2 & 3 \\ 0 & 0 & -2 & 1 \\ 0 & 0 & 0 & 4 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = \begin{bmatrix} -10 \\ 10 \\ 1 \\ 12 \end{bmatrix}$$

(a) by row-oriented back substitution, (b) by column-oriented back substitution. □

### Block Algorithms

It is easy to develop block variants of both forward and back substitution. We will focus on forward substitution. Suppose the lower triangular matrix $G$ has been partitioned into blocks as follows:

$$G = \begin{array}{c} \\ r_1 \\ r_2 \\ \vdots \\ r_s \end{array} \begin{array}{c} \begin{array}{cccc} r_1 & r_2 & \cdots & r_s \end{array} \\ \begin{bmatrix} G_{11} & 0 & \cdots & 0 \\ G_{21} & G_{22} & & 0 \\ \vdots & \vdots & \ddots & \vdots \\ G_{s1} & G_{s2} & \cdots & G_{ss} \end{bmatrix} \end{array}.$$

Each $G_{ii}$ is square and lower triangular. Then the equation $Gy = b$ can be written in the partitioned form

$$\begin{bmatrix} G_{11} & 0 & \cdots & 0 \\ G_{21} & G_{22} & & 0 \\ \vdots & \vdots & \ddots & \vdots \\ G_{s1} & G_{s2} & \cdots & G_{ss} \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_s \end{bmatrix} = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_s \end{bmatrix}. \tag{1.3.18}$$

In this equation the entries $b_i$ and $y_i$ are not scalars; they are vectors with $r_i$ components each. (The partition (1.3.8) is a special case of (1.3.18), and so is the partition used in Exercise 1.3.29 below.) Equation (1.3.18) suggests that we find $y_1$ by solving the system $G_{11}y_1 = b_1$. Once we have $y_1$, we can solve the equation $G_{21}y_1 + G_{22}y_2 = b_2$ for $y_2$, and so on. This leads to the block version of row-oriented forward substitution:

$$\begin{array}{l} \text{for } i = 1, \ldots, s \\ \quad \left[ \begin{array}{l} \text{for } j = 1, \ldots, i-1 \text{ (not executed when } i = 1) \\ \quad \left[ \; b_i \leftarrow b_i - G_{ij}b_j \right. \\ \text{if } G_{ii} \text{ is singular, set error flag, exit} \\ b_i \leftarrow G_{ii}^{-1} b_i \end{array} \right. \end{array} \tag{1.3.19}$$

This is nearly identical to (1.3.5). The operation $b_i \leftarrow G_{ii}^{-1}b_i$ does not require explicit computation of $G_{ii}^{-1}$. It can be effected by solving the lower-triangular system $G_{ii}x = b_i$ by either row- or column-oriented forward substitution.

**Exercise 1.3.20** Write the block variant of the column-oriented forward substitution algorithm (1.3.13). □

**Exercise 1.3.21** Convince yourself that the block versions of forward substitution perform exactly the same arithmetic as the scalar algorithms (1.3.5) and (1.3.13), but not in the same order. □

### Additional Exercises

**Exercise 1.3.22** Write Fortran subroutines to do each of the following: (a) row-oriented forward substitution, (b) column-oriented forward substitution, (c) row-oriented back substitution, (d) column-oriented back substitution. Invent some problems on which to test your programs.

An easy way to devise a problem $Ax = b$ with a known solution is to specify the matrix $A$ and the solution $x$, then multiply $A$ by $x$ to get $b$. Give $A$ and $b$ to your program, and see if it can calculate $x$. □

**Exercise 1.3.23** Prove that if $G$ is triangular, then $\det(G) = g_{11}g_{22}\cdots g_{nn}$. □

**Exercise 1.3.24** Devise a proof of Theorem 1.3.1 that does not use determinants. For example, use condition (c) or (d) of Theorem 1.2.3 instead. □

**Exercise 1.3.25** Write down the numbers $1, 2, 3, \ldots, n-1$ on a line. Immediately below these, write down the numbers $n-1, n-2, n-3, \ldots, 1$. Add these numbers up by summing column-wise first. Conclude that

$$2 \times (1 + 2 + 3 + \cdots + n - 1) = n(n-1).$$

□

**Exercise 1.3.26** In this exercise you will use mathematical induction to draw the same conclusion as in the previous exercise. If you are weak on mathematical induction, you should definitely work this exercise. We wish to prove that

$$\sum_{i=1}^{n-1} i = \frac{n(n-1)}{2} \tag{1.3.27}$$

for all positive integers $n$. Begin by showing that (1.3.27) holds when $n = 1$. The sum on the left-hand side is empty in this case. If you feel nervous about this, you can check the case $n = 2$ as well. Next show that if (1.3.27) holds for $n = k$, then it holds also holds for $n = k + 1$. That is, show that

$$\sum_{i=1}^{k} i = \frac{(k+1)k}{2}$$

is true, assuming that

$$\sum_{i=1}^{k-1} i = \frac{k(k-1)}{2}$$

is true. This is just a matter of simple algebra. Once you have done this, you will have proved by induction that (1.3.27) holds for all positive integers $n$. □

**Exercise 1.3.28** This exercise introduces a useful approximation technique. Draw pictures that demonstrate the inequalities

$$\int_0^{n-1} x\,dx \leq \sum_{i=1}^{n-1} i \leq \int_0^{n} x\,dx.$$

Evaluate the integrals and deduce that

$$\sum_{i=1}^{n-1} i = \frac{n^2}{2} + O(n) \approx \frac{n^2}{2}.$$

Show that the same result holds for $\sum_{i=1}^{n} i$. □

**Exercise 1.3.29** We derived the column-oriented version of forward substitution by partitioning the system $Gy = b$. Different partitions lead to different versions. For example, consider the partition

$$\begin{bmatrix} \hat{G} & 0 \\ h^T & g_{nn} \end{bmatrix} \begin{bmatrix} \hat{y} \\ y_n \end{bmatrix} = \begin{bmatrix} \hat{b} \\ b_n \end{bmatrix},$$

where $\hat{G}$ is $(n-1) \times (n-1)$.

(a) Derive a recursive algorithm based on this partition.

(b) Write a nonrecursive version of the algorithm. (*Hint:* Think about how your recursive algorithm would calculate $y_i$, given $y_1, \ldots, y_{i-1}$.)

Observe that your nonrecursive algorithm is nothing but row-oriented forward substitution. □

## 1.4 POSITIVE DEFINITE SYSTEMS; CHOLESKY DECOMPOSITION

In this section we discuss the problem of solving systems of linear equations for which the coefficient matrix is of a special form, namely, positive definite. If you would prefer to read about general systems first, you can skip ahead to Sections 1.7 and 1.8.

Recall that the *transpose* of an $n \times m$ matrix $A = (a_{ij})$, denoted $A^T$, is the $m \times n$ matrix whose $(i,j)$ entry is $a_{ji}$. Thus the rows of $A^T$ are the columns of $A$. A square matrix $A$ is *symmetric* if $A = A^T$, that is, $a_{ij} = a_{ji}$ for all $i$ and $j$. The matrices

in the electrical circuit examples of Section 1.2 are all symmetric, as are those in the examples of mass-spring systems. The matrix in Example 1.2.12 (approximation of a differential equation) is not symmetric unless $c$ (convection coefficient) is zero.

Since every vector is also a matrix, every vector has a transpose: A column vector $x$ is a matrix with one column. Its transpose $x^T$ is a matrix with one row. The set of column $n$-vectors with real entries will be denoted $\mathbb{R}^n$. That is, $\mathbb{R}^n$ is just the set of real $n \times 1$ matrices.

If $A$ is $n \times n$, real, symmetric, and also satisfies the property

$$x^T Ax > 0 \tag{1.4.1}$$

for all nonzero $x \in \mathbb{R}^n$, then $A$ is said to be *positive definite*.[3] The left-hand side of (1.4.1) is a matrix product. Examining the dimensions of $x^T$, $A$, and $x$, we find that $x^T Ax$ is a $1 \times 1$ matrix, that is, a real number. Thus (1.4.1) is just an inequality between real numbers. It is also possible to define complex positive definite matrices. See Exercises 1.4.63 through 1.4.65.

Positive definite matrices arise frequently in applications. Typically the expression $x^T Ax$ has physical significance. For example, the matrices in the electrical circuit problems of Section 1.2 are all positive definite. In the examples in which the entries of $x$ are loop currents, $x^T Ax$ turns out to be the total power drawn by the resistors in the circuit (Exercise 1.4.66). This is clearly a positive quantity. In the examples in which the entries of $x$ are nodal voltages, $x^T Ax$ is closely related to (and slightly exceeds) the power drawn by the circuit (Exercise 1.4.67).

The matrices of the mass-spring systems in Section 1.2 are also positive definite. In those systems $\frac{1}{2}x^T Ax$ is the *strain energy* of the system, the energy stored in the springs due to compression or stretching (Exercise 1.4.68). This is a positive quantity.

Other areas in which positive definite matrices arise are least-squares problems (Chapter 3), statistics (Exercise 1.4.69), and the numerical solution of partial differential equations (Chapter 7).

**Theorem 1.4.2** *If $A$ is positive definite, then $A$ is nonsingular.*

**Proof.** We will prove the contrapositive form of the theorem: If $A$ is singular, then $A$ is not positive definite. Assume $A$ is singular. Then by Theorem 1.2.3, part (b), there is a nonzero $y \in \mathbb{R}^n$ such that $Ay = 0$. But then $y^T Ay = 0$, so $A$ is not positive definite. □

**Corollary 1.4.3** *If $A$ is positive definite, the linear system $Ax = b$ has exactly one solution.*

**Theorem 1.4.4** *Let $M$ be any $n \times n$ nonsingular matrix, and let $A = M^T M$. Then $A$ is positive definite.*

[3] Some books, notably [33], do not include symmetry as part of the definition.

**Proof.** First we must show that $A$ is symmetric. Recalling the elementary formulas $(BC)^T = C^T B^T$ and $B^{TT} = B$, we find that $A^T = (M^T M)^T = M^T M^{TT} = M^T M = A$. Next we must show that $x^T Ax > 0$ for all nonzero $x$. For any such $x$, we have $x^T Ax = x^T M^T Mx$. Let $y = Mx$, so that $y^T = x^T M^T$. Then $x^T Ax = y^T y = \sum_{i=1}^{n} y_i^2$. This sum of squares is certainly nonnegative, and it is strictly positive if $y \neq 0$. But clearly $y = Mx \neq 0$, because $M$ is nonsingular, and $x$ is nonzero. Thus $x^T Ax > 0$ for all nonzero $x$, and $A$ is positive definite. □

Theorem 1.4.4 provides an easy means of constructing positive definite matrices: Just multiply any nonsingular matrix by its transpose.

**Example 1.4.5** Let $M = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$. $M$ is nonsingular, since $\det(M) = -2$. Therefore $A = M^T M = \begin{bmatrix} 10 & 14 \\ 14 & 20 \end{bmatrix}$ is positive definite. □

**Example 1.4.6** Let

$$M = \begin{bmatrix} 1 & 1 & 1 \\ 0 & 1 & 1 \\ 0 & 0 & 1 \end{bmatrix}.$$

$M$ is nonsingular, since $\det(M) = 1$. Therefore

$$A = M^T M = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 2 & 2 \\ 1 & 2 & 3 \end{bmatrix}$$

is positive definite. □

The next theorem, the Cholesky Decomposition Theorem, is the most important result of this section. It states that every positive definite matrix is of the form $M^T M$ for some $M$. Thus the recipe given by Theorem 1.4.4 generates all positive definite matrices. Furthermore $M$ can be chosen to have a special form.

**Theorem 1.4.7** *(Cholesky Decomposition Theorem) Let $A$ be positive definite. Then $A$ can be decomposed in exactly one way into a product*

$$A = R^T R \qquad \text{(Cholesky Decomposition)}$$

*such that $R$ is upper triangular and has all main diagonal entries $r_{ii}$ positive. $R$ is called the* Cholesky factor *of $A$.*

The theorem will be proved later in the section. Right now it is more important to discuss how the Cholesky decomposition can be used and figure out how to compute the Cholesky factor.

**Example 1.4.8** Let

$$A = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 2 & 2 \\ 1 & 2 & 3 \end{bmatrix} \quad \text{and} \quad R = \begin{bmatrix} 1 & 1 & 1 \\ 0 & 1 & 1 \\ 0 & 0 & 1 \end{bmatrix}.$$

$R$ is upper triangular and has positive main-diagonal entries. In Example 1.4.6 we observed that $A = R^T R$. Therefore $R$ is the Cholesky factor of $A$. □

The Cholesky decomposition is useful because $R$ and $R^T$ are triangular. Suppose we wish to solve the system $Ax = b$, where $A$ is positive definite. If we know the Cholesky factor $R$, we can write the system as $R^T Rx = b$. Let $y = Rx$. We do not know $x$, so we do not know $y$ either. However, $y$ clearly satisfies $R^T y = b$. Since $R^T$ is lower triangular, we can solve for $y$ by forward substitution. Once we have $y$, we can solve the upper-triangular system $Rx = y$ for $x$ by back substitution. The total flop count is a mere $2n^2$, if we know the Cholesky factor $R$.

If the Cholesky decomposition is to be a useful tool, we must find a practical method for calculating the Cholesky factor. One of the easiest ways to do this is to write out the decomposition $A = R^T R$ in detail and study it:

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} & \cdots & a_{1n} \\ a_{21} & a_{22} & a_{23} & \cdots & a_{2n} \\ a_{31} & a_{32} & a_{33} & \cdots & a_{3n} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & a_{n3} & \cdots & a_{nn} \end{bmatrix}$$

$$= \begin{bmatrix} r_{11} & 0 & 0 & \cdots & 0 \\ r_{12} & r_{22} & 0 & \cdots & 0 \\ r_{13} & r_{23} & r_{33} & & 0 \\ \vdots & \vdots & \vdots & \ddots & \\ r_{1n} & r_{2n} & r_{3n} & \cdots & r_{nn} \end{bmatrix} \begin{bmatrix} r_{11} & r_{12} & r_{13} & \cdots & r_{1n} \\ 0 & r_{22} & r_{23} & \cdots & r_{2n} \\ 0 & 0 & r_{33} & \cdots & r_{3n} \\ \vdots & \vdots & & \ddots & \vdots \\ 0 & 0 & 0 & & r_{nn} \end{bmatrix}.$$

The element $a_{ij}$ is the (inner) product of the $i$th row of $R^T$ with the $j$th column of $R$. Noting that the first row of $R^T$ has only one nonzero entry, we focus on this row:

$$a_{1j} = r_{11}r_{1j} + 0r_{2j} + 0r_{3j} + \cdots + 0r_{nj} = r_{11}r_{1j}.$$

In particular, when $j = 1$ we have $a_{11} = r_{11}^2$, which tells us that

$$r_{11} = +\sqrt{a_{11}}. \tag{1.4.9}$$

We know that the positive square root is the right one, because the main-diagonal entries of $R$ are positive. Now that we know $r_{11}$, we can use the equation $a_{1j} = r_{11}r_{1j}$ to calculate the rest of the first row of $R$:

$$r_{1j} = a_{1j}/r_{11}, \quad j = 2, \ldots, n. \tag{1.4.10}$$

This is also the first column of $R^T$. We next focus on the second row, because the second row of $R^T$ has only two nonzero entries. We have

$$a_{2j} = r_{12}r_{1j} + r_{22}r_{2j}. \tag{1.4.11}$$

Only elements from the first two rows of $R$ appear in this equation. In particular, when $j = 2$ we have $a_{22} = r_{12}^2 + r_{22}^2$. Since $r_{12}$ is already known, we can use this

equation to calculate $r_{22}$:

$$r_{22} = +\sqrt{a_{22} - r_{12}^2}.$$

Once $r_{22}$ is known, the only unknown left in (1.4.11) is $r_{2j}$. Thus we can use (1.4.11) to compute the rest of the second row of $R$:

$$r_{2j} = (a_{2j} - r_{12}r_{1j})/r_{22}, \quad j = 3, \ldots, n.$$

There is no need to calculate $r_{21}$ because $r_{21} = 0$. We now know the first two rows of $R$ (and the first two columns of $R^T$). Now, as an exercise, you can show how to calculate the third row of $R$.

Now let us see how to calculate the $i$th row of $R$, assuming that we already have the first $i-1$ rows. Since only the first $i$ entries in the $i$th row of $R^T$ are nonzero,

$$a_{ij} = r_{1i}r_{1j} + r_{2i}r_{2j} + \cdots + r_{i-1,i}r_{i-1,j} + r_{ii}r_{ij}. \tag{1.4.12}$$

All entries of $R$ appearing in (1.4.12) lie in the first $i$ rows. Since the first $i-1$ rows are known, the only unknowns in (1.4.12) are $r_{ii}$ and $r_{ij}$. Taking $i = j$ in (1.4.12), we have

$$a_{ii} = r_{1i}^2 + r_{2i}^2 + \cdots + r_{i-1,i}^2 + r_{ii}^2,$$

which we can solve for $r_{ii}$:

$$r_{ii} = +\sqrt{a_{ii} - \sum_{k=1}^{i-1} r_{ki}^2}. \tag{1.4.13}$$

Now that we have $r_{ii}$, we can use (1.4.12) to solve for $r_{ij}$:

$$r_{ij} = \left(a_{ij} - \sum_{k=1}^{i-1} r_{ki}r_{kj}\right) \Big/ r_{ii} \qquad j = i+1, \ldots, n. \tag{1.4.14}$$

We do not have to calculate $r_{ij}$ for $j < i$ because those entries are all zero.

Equations (1.4.13) and (1.4.14) give a complete recipe for calculating $R$. They even hold for the first row of $R$ ($i = 1$) if we make use of our convention that the sums $\sum_{k=1}^{0}$ are zero. Notice also that when $i = n$, nothing is done in (1.4.14); the only nonzero entry in the $n$th row of $R$ is $r_{nn}$.

The algorithm we have just developed is called *Cholesky's method*. This, the first of several formulations that we will derive, is called the *inner-product formulation* because the sums in (1.4.13) and (1.4.14) can be regarded as inner products. Cholesky's method turns out to be closely related to the familiar Gaussian elimination method. The connection between them is established in Section 1.7.

A number of important observations can now be made. First, recall that the Cholesky decomposition theorem (which we haven't proved yet) makes two assertions: (i) $R$ exists, and (ii) $R$ is unique. In the process of developing the inner-product form of Cholesky's method, we have proved that $R$ is unique: The equation $A = R^T R$ and the stipulation that $R$ is upper triangular with $r_{11} > 0$ imply (1.4.9). Thus this

value of $r_{11}$ is the only one that will work; $r_{11}$ is uniquely determined. Similarly, $r_{1j}$ is uniquely determined by (1.4.10) for $j = 2, \ldots, n$. Thus the first row of $R$ is uniquely determined. Now suppose the first $i-1$ rows are uniquely determined. Then so is the $i$th row, for $r_{ii}$ is uniquely specified by (1.4.13), and $r_{ij}$ is uniquely determined by (1.4.14) for $j = i+1, \ldots, n$. Notice the importance of the stipulation $r_{ii} > 0$ in determining which square root to choose. Without this stipulation we would not have uniqueness.

**Exercise 1.4.15** Let $A = \begin{bmatrix} 4 & 0 \\ 0 & 9 \end{bmatrix}$. (a) Prove that $A$ is positive definite. (b) Calculate the Cholesky factor of $A$. (c) Find three other upper triangular matrices $R$ such that $A = R^T R$. (d) Let $A$ be any $n \times n$ positive definite matrix. How many upper-triangular matrices $R$ such that $A = R^T R$ are there? □

The next important observation is that Cholesky's method serves as a test of positive definiteness. By Theorems 1.4.4 and 1.4.7, $A$ is positive definite if and only if there exists an upper triangular matrix $R$ with positive main diagonal entries such that $A = R^T R$. Given any symmetric matrix $A$, we can attempt to calculate $R$ by Cholesky's method. If $A$ is not positive definite, the algorithm must fail, because any $R$ that satisfies (1.4.13) and (1.4.14) must also satisfy $A = R^T R$. The algorithm fails if and only if at some step the number under the square root sign in (1.4.13) is negative or zero. In the first case there is no real square root; in the second case $r_{ii} = 0$. Thus, if $A$ is not positive definite, there must come a step at which the algorithm attempts to take the square root of a number that is not positive. Conversely, if $A$ is positive definite, the algorithm cannot fail. The equation $A = R^T R$ guarantees that the number under the square root sign in (1.4.13) is positive at every step. (After all, it equals $r_{ii}^2$.) Thus Cholesky's method succeeds if and only if $A$ is positive definite. This is the best general test of positive definiteness known.

The next thing to notice is that (1.4.13) and (1.4.14) use only those $a_{ij}$ for which $i \leq j$. This is not surprising, since in a symmetric matrix the entries above the main diagonal are identical to those below the main diagonal. This underscores the fact that in a computer program we do not need to store all of $A$; there is no point in duplicating information. If space is at a premium, the programmer may choose to store $A$ in a long one-dimensional array with $a_{11}, a_{12}, \ldots, a_{1n}$ immediately followed by $a_{22}, a_{23}, \ldots, a_{2n}$, immediately followed by $a_{33}, \ldots, a_{3n}$, and so on. This compact storage scheme makes the programming more difficult but is worth using if space is scarce.

Finally we note that each element $a_{ij}$ is used only to compute $r_{ij}$, as is clear from (1.4.13) and (1.4.14). It follows that in a computer program, $r_{ij}$ can be stored over $a_{ij}$. This saves additional space by eliminating the need for separate arrays to store $R$ and $A$.

**Exercise 1.4.16** Write an algorithm based on (1.4.13) and (1.4.14) that checks a matrix for positive definiteness and calculates $R$, storing $R$ over $A$. □

Your solution to Exercise 1.4.16 should look something like this:

**Cholesky's Algorithm (inner-product form)**

```
for i = 1,...,n
  [ for k = 1,...,i-1 (not executed when i = 1)
      [ a_ii ← a_ii - a_ki^2
    if a_ii ≤ 0, set error flag (A is not positive definite), exit
    a_ii ← sqrt(a_ii) (this is r_ii)
    for j = i+1,...,n (not executed when i = n)
      [ for k = 1,...,i-1 (not executed when i = 1)
          [ a_ij ← a_ij - a_ki a_kj
        a_ij ← a_ij / a_ii (this is r_ij)
```

(1.4.17)

The upper part of $R$ is stored over the upper part of $A$. There is no need to store the lower part of $R$ because it consists entirely of zeros.

**Example 1.4.18** Let

$$A = \begin{bmatrix} 4 & -2 & 4 & 2 \\ -2 & 10 & -2 & -7 \\ 4 & -2 & 8 & 4 \\ 2 & -7 & 4 & 7 \end{bmatrix} \quad \text{and} \quad b = \begin{bmatrix} 8 \\ 2 \\ 16 \\ 6 \end{bmatrix}.$$

Notice that $A$ is symmetric. We will use Cholesky's method to show that $A$ is positive definite and calculate the Cholesky factor $R$. We will then use the Cholesky factor to solve the system $Ax = b$ by forward and back substitution.

$$\begin{aligned}
r_{11} &= \sqrt{a_{11}} = \sqrt{4} = 2 \\
r_{12} &= a_{12}/r_{11} = -2/2 = -1 \\
r_{13} &= 2 \\
r_{14} &= 1 \\
r_{22} &= \sqrt{a_{22} - r_{12}^2} = \sqrt{10 - (-1)^2} = 3 \\
r_{23} &= \frac{a_{23} - r_{13}r_{12}}{r_{22}} = \frac{-2 - (-1)2}{3} = 0 \\
r_{24} &= -2 \\
r_{33} &= \sqrt{a_{33} - r_{13}^2 - r_{23}^2} = \sqrt{8 - 2^2 - 0^2} = 2 \\
r_{34} &= \frac{a_{34} - r_{14}r_{13} - r_{24}r_{23}}{r_{33}} = \frac{4 - (2)(1) - (0)(-2)}{2} = 1 \\
r_{44} &= \sqrt{a_{44} - r_{14}^2 - r_{24}^2 - r_{34}^2} = \sqrt{7 - 1^2 - (-2)^2 - 1^2} = 1.
\end{aligned}$$

Thus

$$R = \begin{bmatrix} 2 & -1 & 2 & 1 \\ 0 & 3 & 0 & -2 \\ 0 & 0 & 2 & 1 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

Since we were able to calculate $R$, $A$ is positive definite. We can check our work by multiplying $R^T$ by $R$ and seeing that the product is $A$.

To solve $Ax = b$, we first solve $R^T y = b$ by forward substitution and obtain $y = \begin{bmatrix} 4 & 2 & 4 & 2 \end{bmatrix}^T$. We then solve $Rx = y$ by back substitution to obtain $x = \begin{bmatrix} 1 & 2 & 1 & 2 \end{bmatrix}^T$. Finally, we check our work by multiplying $A$ by $x$ and seeing that we get $b$. □

**Exercise 1.4.19** Calculate $R$ (of Example 1.4.18) by the *erasure method*. Start with an array that has $A$ penciled in initially (main diagonal and upper triangle only). As you calculate each entry $r_{ij}$, erase $a_{ij}$ and replace it by $r_{ij}$. Do all of the operations in your head, using only the single array for reference. This procedure is surprisingly easy, once you get the hang of it. □

**Example 1.4.20** Let

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 2 & 5 & 10 \\ 3 & 10 & 16 \end{bmatrix}.$$

We will use Cholesky's method to determine whether or not $A$ is positive definite. Proceeding as in Example 1.4.18, we find that $r_{11} = 1$, $r_{12} = 2$, $r_{13} = 3$, $r_{22} = 1$, $r_{23} = 4$, and finally $r_{33} = \sqrt{a_{33} - r_{13}^2 - r_{23}^2} = \sqrt{-9}$. In attempting to calculate $r_{33}$, we encounter a negative number under the square root sign. Thus $A$ is not positive definite. □

**Exercise 1.4.21** Let

$$A = \begin{bmatrix} 16 & 4 & 8 & 4 \\ 4 & 10 & 8 & 4 \\ 8 & 8 & 12 & 10 \\ 4 & 4 & 10 & 12 \end{bmatrix} \quad \text{and} \quad b = \begin{bmatrix} 32 \\ 26 \\ 38 \\ 30 \end{bmatrix}.$$

Notice that $A$ is symmetric. (a) Use the inner-product formulation of Cholesky's method to show that $A$ is positive definite and compute its Cholesky factor. (b) Use forward and back substitution to solve the linear system $Ax = b$. □

**Exercise 1.4.22** Determine whether or not each of the following matrices is positive definite.

$$A = \begin{bmatrix} 9 & 3 & 3 \\ 3 & 10 & 7 \\ 3 & 5 & 9 \end{bmatrix} \qquad B = \begin{bmatrix} 4 & 2 & 6 \\ 2 & 2 & 5 \\ 6 & 5 & 29 \end{bmatrix}$$

$$C = \begin{bmatrix} 4 & 4 & 8 \\ 4 & -4 & 1 \\ 8 & 1 & 6 \end{bmatrix} \qquad D = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 2 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

□

**Exercise 1.4.23** Rework Exercise 1.4.22 with the help of MATLAB. The MATLAB command `R = chol(A)` computes the Cholesky factor of $A$ and stores it in $R$. The upper

half of $A$ is used in the computation. (MATLAB does not check whether or not $A$ is symmetric. For more details about `chol`, type `help chol`.) □

Although Cholesky's method generally works well, a word of caution is appropriate here. Unlike the small hand computations that are scattered throughout the book, most matrix computations are performed by computer, in which case the arithmetic operations are subject to roundoff errors. In Chapter 2 we will see that the performance of Cholesky's method in the face of roundoff errors is as good as we could hope for. However, there are linear systems, called ill-conditioned systems, that simply cannot be solved accurately in the presence of errors. Naturally we cannot expect Cholesky's method (performed with roundoff errors) to solve ill-conditioned systems accurately. For more on ill-conditioned systems and roundoff errors, see Chapter 2.

### Flop Count

To count the flops in Cholesky's algorithm (1.4.17), we need to know that

$$\sum_{i=1}^{n} i^2 = \frac{n^3}{3} + O(n^2).$$

The easiest way to obtain this is to approximate the sum by an integral:

$$\sum_{i=1}^{n} i^2 \approx \int_0^n i^2 \, di = \frac{n^3}{3}.$$

The details are discussed in Exercises 1.4.70 and 1.4.71.

**Proposition 1.4.24** *Cholesky's algorithm (1.4.17) applied to an $n \times n$ matrix performs about $n^3/3$ flops.*

**Exercise 1.4.25** Prove Proposition 1.4.24 □

**Proof.** Examining (1.4.17), we see that in each of the two $k$ loops, two flops are performed. To see how many times each loop is executed, we look at the limits on the loop indices. We conclude that the number of flops attributable to the first of the $k$ loops is

$$\sum_{i=1}^{n} \sum_{k=1}^{i-1} 2 = 2 \sum_{i=1}^{n} (i-1) = n(n-1) \approx n^2$$

by Exercise 1.3.25 or 1.3.26. Applying the same procedure to the second of the $k$ loops, we get a flop count of

$$\sum_{i=1}^{n} \sum_{j=i+1}^{n} \sum_{k=1}^{i-1} 2.$$

We have a triple sum this time, because the loops are nested three deep.

$$\begin{aligned}
\sum_{i=1}^{n}\sum_{j=i+1}^{n}\sum_{k=1}^{i-1}2 &= 2\sum_{i=1}^{n}\sum_{j=i+1}^{n}(i-1) \\
&= 2\sum_{i=1}^{n}(n-i)(i-1) \\
&= 2n\sum_{i=1}^{n}(i-1) - 2\sum_{i=1}^{n}i^2 + 2\sum_{i=1}^{n}i \\
&= n^3 - 2\frac{n^3}{3} + O(n^2) \\
&\approx \frac{n^3}{3}.
\end{aligned}$$

Here we have used the estimates $2\sum_{i=1}^{n}(i-1) = n^2 + O(n)$ and $2\sum_{i=1}^{n} i = n^2 + O(n)$. In the end we discard the $O(n^2)$ term, because it is small in comparison with the term $n^3/3$, once $n$ is sufficiently large. Thus about $n^3/3$ flops are performed in the second $k$ loop. Notice that the number of flops performed in the first $k$ loop is negligible by comparison.

In addition to the flops in the $k$ loops, there are some divisions. The exact number is

$$\sum_{i=1}^{n}\sum_{j=i+1}^{n}1 = \sum_{i=1}^{n}(n-i) = \frac{n(n-1)}{2},$$

which is also negligible. Finally, $\sum_{i=1}^{n}1 = n$ error checks and square roots are done. We conclude that the flop count for (1.4.17) is $n^3/3 + O(n^2)$. □

Since the flop count is $O(n^3)$, we expect that each time we double the matrix dimension, the time it takes to compute the Cholesky factor will be multiplied by about eight. See Exercise 1.4.72.

If we wish to solve a system $Ax = b$ by Cholesky's method, we must first compute the Cholesky decomposition at a cost of about $n^3/3$ flops. Then we must perform forward and back substitution using the Cholesky factor and its transpose at a total cost of about $2n^2$ flops. We conclude that the bulk of the time is spent computing the Cholesky factor; the forward and backward substitution times are negligible. Thus the cost of solving a large system using Cholesky's method can be reckoned to be $n^3/3$ flops. Each time we double the dimension of the system, we can expect the time it takes to solve $Ax = b$ by Cholesky's method to be multiplied by about eight.

### Outer-Product Form of Cholesky's Method

The *outer-product* form of Cholesky's method is derived by partitioning the equation $A = R^T R$ in the form

$$\begin{bmatrix} a_{11} & b^T \\ b & \hat{A} \end{bmatrix} = \begin{bmatrix} r_{11} & 0 \\ s & \hat{R}^T \end{bmatrix} \begin{bmatrix} r_{11} & s^T \\ 0 & \hat{R} \end{bmatrix}. \tag{1.4.26}$$

Equating the blocks, we obtain

$$a_{11} = r_{11}^2, \qquad b^T = r_{11} s^T, \qquad \hat{A} = ss^T + \hat{R}^T \hat{R}. \tag{1.4.27}$$

The fourth equation, $b = sr_{11}$, is redundant. Equations (1.4.27) suggest the following procedure for calculating $r_{11}$, $s^T$, and $\hat{R}$ (and hence $R$):

$$\begin{aligned} r_{11} &= \sqrt{a_{11}} \\ s^T &= r_{11}^{-1} b^T \\ \tilde{A} &= \hat{A} - ss^T \\ \text{Solve} \quad \tilde{A} &= \hat{R}^T \hat{R} \quad \text{for } \hat{R}. \end{aligned} \tag{1.4.28}$$

This procedure reduces the $n \times n$ problem to that of finding the Cholesky factor of the $(n-1) \times (n-1)$ matrix $\tilde{A}$. This problem can be reduced to an $(n-2) \times (n-2)$ problem by the same algorithm, and so on. Eventually the problem is reduced to the trivial $1 \times 1$ case. This is called the *outer-product* formulation because at each step an outer product $ss^T$ is subtracted from the remaining submatrix. It can be implemented recursively or nonrecursively with no difficulty.

**Exercise 1.4.29** Use the outer-product formulation of Cholesky's method to calculate the Cholesky factor of the matrix of Example 1.4.18. □

**Exercise 1.4.30** Use the outer-product formulation of Cholesky's method to work Example 1.4.20. □

**Exercise 1.4.31** Use the outer-product form to work part (a) of Exercise 1.4.21. □

**Exercise 1.4.32** Use the outer-product form to work Exercise 1.4.22. □

**Exercise 1.4.33** Write a nonrecursive algorithm that implements the outer-product formulation of Cholesky's algorithm (1.4.28). Your algorithm should exploit the symmetry of $A$ by referencing only the main diagonal and upper part of $A$, and it should store $R$ over $A$. Be sure to put in the necessary check before taking the square root. □

**Exercise 1.4.34** (a) Do a flop count for the outer-product formulation of Cholesky's method. You will find that approximately $n^3/3$ flops are performed, the same number as for the inner-product formulation. (If you do an exact flop count, you will find that the counts are exactly equal.) (b) Convince yourself that the outer-product and inner-product formulations of the Cholesky algorithm perform exactly the same operations, but not in the same order. □

### Bordered Form of Cholesky's Method

The bordered form will prove useful in the next section, where we develop a version of Cholesky's method for banded matrices. We start by introducing some new notation and terminology. For $j = 1, \ldots, n$ let $A_j$ be the $j \times j$ submatrix of $A$ consisting of the intersection of the first $j$ rows and columns of $A$. $A_j$ is called the $j$th *leading principal submatrix* of $A$. In Exercise 1.4.54 you will show that if $A$ is positive definite, then all of its leading principal submatrices are positive definite. Suppose $A$ is positive definite, and let $R$ be the Cholesky factor of $A$. Then $R$ has leading principal submatrices $R_j$, $j = 1, \ldots, n$, which are upper triangular and have positive entries on the main diagonal.

**Exercise 1.4.35** By partitioning the equation $A = R^T R$ appropriately, show that $R_j$ is the Cholesky factor of $A_j$ for $j = 1, \ldots, n$. □

It is easy to construct $R_1 = [r_{11}]$, since $a_{11} = r_{11}^2$. Thinking inductively, if we can figure out how to construct $R_j$, given $R_{j-1}$, then we will be able to construct $R_n = R$ in $n$ steps. Suppose therefore that we have calculated $R_{j-1}$ and wish to find $R_j$. Partitioning the equation $A_j = R_j^T R_j$ so that $A_{j-1}$ and $R_{j-1}$ appear as blocks:

$$\begin{bmatrix} A_{j-1} & c \\ c^T & a_{jj} \end{bmatrix} = \begin{bmatrix} R_{j-1}^T & 0 \\ h^T & r_{jj} \end{bmatrix} \begin{bmatrix} R_{j-1} & h \\ 0 & r_{jj} \end{bmatrix}, \tag{1.4.36}$$

we get the equations

$$A_{j-1} = R_{j-1}^T R_{j-1}, \qquad c = R_{j-1}^T h, \qquad a_{jj} = h^T h + r_{jj}^2. \tag{1.4.37}$$

Since we already have $R_{j-1}$, we have only to calculate $h$ and $r_{jj}$ to get $R_j$. Equations (1.4.37) show how this can be done. First solve the equation $c = R_{j-1}^T h$ for $h$ by forward substitution. Then calculate $r_{jj} = \sqrt{a_{jj} - h^T h}$. The algorithm that can be built along these lines is called the *bordered form* of Cholesky' method.

**Exercise 1.4.38** Use the bordered form of Cholesky's method to calculate the Cholesky factor of the matrix of Example 1.4.18. □

**Exercise 1.4.39** Use the bordered form of Cholesky's method to work Example 1.4.20. □

**Exercise 1.4.40** Use the bordered form to work part (a) of Exercise 1.4.21. □

**Exercise 1.4.41** Use the bordered form to work Exercise 1.4.22. □

**Exercise 1.4.42** (a) Do a flop count for the bordered form of Cholesky's method. Again you will find that approximately $n^3/3$ flops are done. (b) Convince yourself that this algorithm performs exactly the same arithmetic operations as the other two formulations of Cholesky's method. □

We have now introduced three different versions of Cholesky's method. We have observed that the inner-product formulation (1.4.17) has triply nested loops; so do

the others. If one examines all of the possibilities, one finds that there are six distinct basic variants, associated with the six (= 3!) different ways of nesting three loops. Exercise 1.7.55 discusses the six variants.

### Computing the Cholesky Decomposition by Blocks

All formulations of the Cholesky decomposition algorithm have block versions. As we have shown in Section 1.1, block implementations can have superior performance on large matrices because of more efficient use of cache and greater ease of parallelization.

Let us develop a block version of the outer-product form. Generalizing (1.4.26), we write the equation $A = R^T R$ by blocks:

$$\begin{bmatrix} A_{11} & B \\ B^T & \hat{A} \end{bmatrix} = \begin{bmatrix} R_{11}^T & 0 \\ S^T & \hat{R}^T \end{bmatrix} \begin{bmatrix} R_{11} & S \\ 0 & \hat{R} \end{bmatrix}.$$

$A_{11}$ and $R_{11}$ are square matrices of dimension $d_1 \times d_1$, say. $A_{11}$ is symmetric and (by Exercise 1.4.54) positive definite. By the way, this equation also generalizes (1.4.36). Equating the blocks, we have the equations

$$A_{11} = R_{11}^T R_{11}, \quad B = R_{11}^T S, \quad \hat{A} = S^T S + \hat{R}^T \hat{R},$$

which suggest the following procedure for calculating $R$.

$$\begin{aligned} R_{11} &= \text{cholesky}(A_{11}) \\ S &= R_{11}^{-T} B \\ \tilde{A} &= \hat{A} - S^T S \\ \hat{R} &= \text{cholesky}(\tilde{A}). \end{aligned} \tag{1.4.43}$$

The symbol $R_{11}^{-T}$ means $(R_{11}^{-1})^T$, which is the same as $(R_{11}^T)^{-1}$. The operation $S = R_{11}^{-T} B$ does not require the explicit computation of $R_{11}^{-T}$. Instead we can refer back to the equation $R_{11}^T S = B$. Letting $s$ and $b$ denote the $i$th columns of $S$ and $B$, respectively, we see that $R_{11}^T s = b$. Since $R_{11}^T$ is lower triangular, we can obtain $s$ from $b$ by forward substitution. (Just such an operation is performed in the bordered form of Cholesky's method.) Performing this operation for each column of $S$, we obtain $S$ from $B$. This is how the operation $S = R_{11}^{-T} B$ should normally be carried out.

**Exercise 1.4.44** Let $C$ be any nonsingular matrix. Show that $(C^{-1})^T = (C^T)^{-1}$. □

The matrices $\hat{A}$, $B$, $\hat{R}$, and $S$ may themselves be partitioned into blocks. Consider a finer partition of $A$:

$$A = \begin{array}{c} \\ d_1 \\ d_2 \\ \vdots \\ d_s \end{array} \begin{array}{c} \begin{array}{cccc} d_1 & d_2 & \cdots & d_s \end{array} \\ \begin{bmatrix} A_{11} & A_{12} & \cdots & A_{1s} \\ & A_{22} & \cdots & A_{2s} \\ & & \ddots & \vdots \\ & & & A_{ss} \end{bmatrix} \end{array}.$$

We have shown only the upper half because of symmetry. Then

$$B = \begin{bmatrix} A_{12} & \cdots & A_{1s} \end{bmatrix},$$

and the operation $S = R_{11}^{-T} B$ becomes

$$S_{1j} = R_{11}^{-T} A_{1j}, \qquad j = 2, \ldots, s,$$

where we partition $R$ conformably with $A$, and the operation $\tilde{A} = \hat{A} - S^T S$ becomes

$$\tilde{A}_{ij} = \hat{A}_{ij} - R_{1i}^T R_{1j}, \qquad i, j = 1, \ldots, s.$$

Once we have $\tilde{A}$, we can calculate its Cholesky factor by applying (1.4.43) to it.

**Exercise 1.4.45** Write a nonrecursive algorithm that implements the algorithm that we have just sketched. Your algorithm should exploit the symmetry of $A$ by referencing only the main diagonal and upper part of $A$, and it should store $R$ over $A$. □

Your solution to Exercise 1.4.45 should look something like this:

**Block Cholesky Algorithm (outer-product form)**

```
for k = 1, ..., s
    A_kk ← cholesky(A_kk)   (if cholesky fails, set flag and exit)
    for j = k+1, ..., s
        A_kj ← A_kk^{-T} A_kj                                    (1.4.46)
    for i = k+1, ..., s
        for j = i, ..., s
            A_ij ← A_ij − A_ki^T A_kj
```

In order to implement this algorithm, we need a standard Cholesky decomposition code (based on (1.4.17), for example) to perform the small Cholesky decompositions $A_{kk} \leftarrow \text{cholesky}(A_{kk})$. In the operation $A_{kj} \leftarrow A_{kk}^{-T} A_{kj}$, the block $A_{kk}$ holds the triangular matrix $R_{kk}$ at this point. Thus the operation can be effected by a sequence of forward substitutions, as already explained; there is no need to calculate an inverse.

**Exercise 1.4.47** Write a block version of the inner-product form of Cholesky's method. □

**Exercise 1.4.48** Convince yourself that the block versions of Cholesky's method perform exactly the same arithmetic as the standard versions, but not in the same order. □

The benefits of organizing the Cholesky decomposition by blocks are exactly the same as those of performing matrix multiplication by blocks, as discussed in Section 1.1. To keep the discussion simple, let us speak as if all of the blocks were square and of the same size, $d \times d$. The bulk of the work is concentrated in the operation

$$A_{ij} \leftarrow A_{ij} - A_{ki}^T A_{kj},$$